MSc Artificial Intelligence
Master Thesis

# Semantic Scene Completion with Latent Modeling of Scene-Level 3D Gaussians

by
Mohammad Hafeez Khan
15549089

January 2026 – May 2026
36 ECTS

Supervisor: Yue Li
University Supervisor: Prof. Martin R. Oswald

Examiner: Dr. Shaodi You



Universiteit van Amsterdam
Instituut voor Informatica

*Semantic Scene Completion with Latent Modeling of Scene-Level 3D Gaussians*

This work by Mohammad Hafeez Khanis licensed under **CC-BY-NC**.
https://creativecommons.org/licenses/by-nc/4.0



This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only.

# Abstract

Photorealistic three-dimensional scene generation has, in the past two years, become both more attractive and more tractable thanks to 3D Gaussian Splatting (3DGS). A natural way to build a generative model on top of this representation is the standard two-stage recipe: first compress scenes into a structured latent with a variational autoencoder, then learn a flow-matching or diffusion model in that latent space. Recent work has shown this is possible for around 8,500 video-captured scenes (Can3Tok, on DL3DV-10K), where each scene's Gaussians are pre-filtered to a clean salient-region subset. Applying the same recipe to several thousand unfiltered, heterogeneous, pre-fitted indoor scenes coming from seven different capture pipelines is what this thesis sets out to study.

I begin from the architecture of Can3Tok (Gao et al., 2025) and use the SceneSplat-7K dataset (Li et al., 2025b) for training. Through a controlled ablation of seven Stage 1 variants, I show that two simple choices control whether the autoencoder generalises or memorises at scale: (1) imposing a fixed-frame Hilbert ordering on the input Gaussians, which makes the slot-aligned reconstruction target identifiable, and (2) replacing the global flat decoder bottleneck with a windowed local encoder paired with a shared per-token MLP. Together they bring validation reconstruction down by more than a factor of two and produce qualitative reconstructions in which spatial separation between objects becomes visible, though far from photorealistic. A small-scale (300-scene) exploration of latent disentanglement and per-Gaussian contrastive supervision motivates a schema-aware Stage 2 design: a hierarchical flow-matching generator with a layout transformer and a geometry transformer for the disentangled variants, and a single scene-level transformer for the flat variants. The trained Stage 2 generator reproduces the structural pattern of the Stage 1 latent well; the completion model recovers coarse layout under partial-scan masking but loses local detail. I am explicit throughout about what the seven-variant ablation supports, about the residual position compression I identified in the Stage 1 decoder output, and about the limits within which the present results should be read.

# Contents

**List of Figures** iv

**List of Tables** v

**1 Introduction** 1
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions . . . 1
- 1.4 Contributions . . . 2
- 1.5 Thesis Structure . . . 2

**2 Related Work** 3
- 2.1 3D Scene Representations . . . 3
- 2.2 Latent Generative Modelling of 3D Content . . . 3
- 2.3 Learning on Unordered Sets and Space-Filling Curves . . . 4
- 2.4 Semantic and Disentangled Representations . . . 4

**3 Methodology** 5
- 3.1 Framework and Problem Setup . . . 5
- 3.2 Data Pipeline . . . 5
  - 3.2.1 Dataset . . . 5
  - 3.2.2 Hilbert Serialisation . . . 5
  - 3.2.3 Colour Decomposition . . . 6
- 3.3 Encoder . . . 6
- 3.4 Reconstruction Objective . . . 8
- 3.5 Decoder . . . 8
  - 3.5.1 Flat vs. Token-Local Decoder . . . 8
  - 3.5.2 Anchor-Relative Position Decoding . . . 8
- 3.6 Disentanglement and the Two Latent Schemas . . . 9
  - 3.6.1 Total Stage 1 Objective . . . 9
- 3.7 Small-Scale Semantic Experiments . . . 9
- 3.8 Stage 2 Generative Model . . . 10
- 3.9 Configuration Summary for the Seven Experiments . . . 11

**4 Results** 13
- 4.1 300-Scene Exploratory Study . . . 13
- 4.2 Seven-Variant Large-Scale Ablation . . . 13
- 4.3 Per-Component Loss Breakdown . . . 14
- 4.4 Validation Convergence . . . 14
- 4.5 Qualitative Stage 1 Reconstruction . . . 14
- 4.6 Per-Gaussian Feature Visualisation . . . 15
- 4.7 Stage 2 Generative Results . . . 15

**5 Discussion** 18
- 5.1 Revisiting the Research Questions . . . 18
- 5.2 What the Unfiltered Heterogeneous Indoor Regime Does to the Loss Landscape . . . 18
- 5.3 Position Compression and What I Tried . . . 19
- 5.4 Limitations . . . 19

**6 Conclusion** 21
- 6.1 Answers to the Research Questions . . . 21
- 6.2 Future Work . . . 21

**A Appendix** **25**
A.1 Hardware and Software Environment . . . . . 25
A.2 Per-Variant Configuration Flags . . . . . 25
A.3 Hyperparameter Defaults . . . . . 25
A.4 Validation Convergence by Variant . . . . . 25
A.5 300-Scene Configurations . . . . . 25
A.6 Reproducibility Notes . . . . . 25

# List of Figures


3.1 Full pipeline overview . . . 6
3.2 Stage 1 VAE architecture detail . . . 7
3.3 Slot ordering stability . . . 7
3.4 Global vs. local encoder . . . 8
3.5 Flat vs. token-local decoder . . . 8
3.6 Stage 2 generation pipeline . . . 11

4.1 Stage 1 reconstructions across four variants . . . 16
4.2 Per-Gaussian feature PCA visualisations . . . 17
4.3 Stage 2 V6 qualitative results . . . 17

# List of Tables

3.1 Configuration matrix for the seven Stage 1 variants . . . 12

4.1 300-scene latent disentanglement comparison . . . 13
4.2 300-scene InfoNCE comparison . . . 13
4.3 Headline numbers for the seven-variant ablation . . . 15
4.4 Per-attribute loss breakdown . . . 15
4.5 Stage 2 training summary . . . 15

A.1 Per-variant SLURM job ids and flags . . . 26
A.2 Common Stage 1 hyperparameters . . . 26
A.3 Combined per-variant validation progression . . . 27

Chapter 1

# Introduction

## 1.1 Motivation

Generating or completing three-dimensional scenes from scratch is one of the more open-ended goals in computer vision and graphics. The downstream applications are wide: rapid content creation for games and film, embodied-AI simulators that need fresh interior layouts, autonomous-driving testing in unseen environments, and robot planning in cluttered spaces. What has changed in the past two years is that the underlying scene representation has matured enough to make this goal look tractable.

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) has rapidly become the default representation for photorealistic scene reconstruction. A scene is modelled as a set of anisotropic Gaussian splats, each carrying a position, an opacity, a colour (the DC spherical-harmonic coefficient), and an anisotropic covariance parameterised by a rotation and a scale. The representation renders in real time, matches or exceeds the quality of earlier implicit methods such as NeRF (Mildenhall et al., 2020), and is explicitly structured: in principle, a network can produce the primitives directly rather than infer them through a coordinate query.

Scene-level 3DGS generation, however, has remained largely unexplored. The standard recipe for generation in adjacent domains is well established. First compress the raw signal into a smooth, compact latent space with a variational autoencoder (Kingma and Welling, 2014), then train a generative model on that latent, with diffusion (Ho et al., 2020) or its successor flow matching (Lipman et al., 2023; Liu et al., 2023) now the most common choice and a transformer backbone such as DiT (Peebles and Xie, 2023) or SiT (Ma et al., 2024) the most common architecture. This recipe is proven at the object level for 3D content (Zeng et al., 2022; Zhao et al., 2023; Zhang et al., 2024), but scaling it to whole scenes is a different problem. A scene contains tens of thousands of unordered Gaussian primitives, spans an unbounded volume, mixes dozens of object categories, and comes with no canonical coordinate frame or natural ordering.

Can3Tok (Gao et al., 2025) provided the first proof of feasibility: it showed that a scene-level 3DGS variational autoencoder could be trained on the DL3DV-10K dataset (Ling et al., 2024) of around 8,500 video-captured outdoor and indoor scenes, with the input Gaussians pre-filtered by a LangSAM-based salient-region selector to retain the cleanest spatially-coherent subset, and that the resulting latent could be used for downstream image-to-3DGS and text-to-3DGS generation. This was the architectural starting point of my thesis. The question I wanted to answer was whether the same recipe could be applied to the indoor scenes of SceneSplat-7K, where the Gaussians come from seven different capture pipelines without any equivalent salient-region filtering, and what would need to change to make that happen. In the early stages of the project the honest answer turned out to be no, not without significant changes. Training the Can3Tok architecture on the SceneSplat-7K dataset (Li et al., 2025b) of nearly 8,000 indoor 3DGS scenes failed to converge in any useful sense: validation reconstruction plateaued, and the train-to-test generalisation gap was large. Identifying what caused this, and how to fix it, is what occupied most of the thesis.

## 1.2 Problem Statement

This thesis studies the construction and training of a semantically aware, scalable, scene-level 3DGS variational autoencoder as Stage 1 of a two-stage generative pipeline for scene generation and completion. The encoder must map a variable-size, unordered Gaussian scene into a fixed-size structured latent $Z \in \mathbb{R}^{T \times d}$, and the decoder must reconstruct the scene from it. Downstream, a flow-matching Stage 2 model is trained on the resulting latent distribution.

The core difficulty I observed empirically, and which the rest of the thesis is structured around, is that the per-primitive parameters of a pre-fitted 3DGS scene are ambiguous in a way that hurts the standard slot-aligned reconstruction objective. Two specific sources of this ambiguity matter.

**No canonical order.** A 3DGS scene is fundamentally a set: any permutation of its elements represents the same scene. A decoder that is asked to reconstruct "the $i$-th Gaussian" is being asked to reconstruct a target that has no stable identity from one scene to the next. Under an element-wise $\ell_2$ loss between predicted and target Gaussians, slot $i$ in scene $A$ and slot $i$ in scene $B$ might correspond to entirely different parts of the room. Without a canonical ordering, the loss has no persistent signal across scenes and the model never converges to anything compositional.

**Less data-side selection.** Can3Tok benefits from a semantic-aware filter that retains the cleanest spatially-coherent subset of each scene's Gaussians before training. In my setting, the splats come from seven indoor capture pipelines (ScanNet, ScanNet++, ARKitScenes, Matterport3D, Replica, Hypersim, 3RScan) with varying view density, and are sampled by top-$N$ opacity ranking without any equivalent salient-region filter. The autoencoder therefore receives a noisier and more heterogeneous set of Gaussians per scene, so each latent token must compress a less-structured input than in Can3Tok's original regime. Both Can3Tok and my setting use pre-fitted frozen splats; the difference is in what gets selected from those splats before training, not in whether the splats are jointly optimised.

Both effects become much more severe at scale. At 300 scenes the model has enough repetition to memorise individual scenes despite these issues. At 3,000 or 6,000 scenes, memorisation is no longer possible, and the issues become the dominant obstacle.

## 1.3 Research Questions

The work addresses one main question and three sub-questions.

**Main question.** *What is required to scale a scene-level 3DGS variational autoencoder from a few hundred training scenes to several thousand heterogeneous scenes, and which design choices matter most?*

**RQ1 (Ordering).** Does imposing a canonical spatial order on the Gaussian set restore convergence at scale, and how should that ordering be combined with the decoder structure?

**RQ2 (Architecture and generalisation).** Does replacing the global encoder bottleneck and the flat decoder MLP with a spatially local encoder and a shared per-token decoder close the train-to-test generalisation gap that appears at scale?

**RQ3 (Semantic structure).** Can per-primitive semantic supervision and a deliberate split between layout and geometry be incorporated into this latent without degrading reconstruction, motivating the design of the Stage 2 generative model?

## 1.4 Contributions

The contributions of this thesis are as follows.

1. **Diagnosis of the scaling failure.** Through a controlled seven-variant ablation, I show that the dominant obstacle to scaling the Can3Tok autoencoder to the heterogeneous unfiltered SceneSplat-7K corpus is the absence of a stable slot-to-primitive correspondence. Without that, the element-wise $\ell_2$ target is not a learnable function and the validation loss saturates regardless of architecture.

2. **Fixed-frame Hilbert serialisation as the fix for ordering.** I serialise each scene's Gaussians along a 3D Hilbert space-filling curve (Skilling, 2004) computed in a fixed canonical frame of radius 10 m. This is the same locality-preserving idea that Point Transformer V3 (Wu et al., 2024) uses for point cloud tokenisation, but applied here directly to the reconstruction target. Because the frame is fixed (not scene-dependent), slot $i$ corresponds to roughly the same physical region across scenes.

3. **Anchor-relative position decoding.** Inspired by Scaffold-GS (Lu et al., 2024) and the structured prediction in GaussianCube (Zhang et al., 2024), I decompose each primitive's predicted position as $\hat{\boldsymbol{\mu}}_i = a_{\tau(i)} + \rho \tanh(o_i)$, where $a_{\tau(i)}$ is the spatial anchor of the token owning Gaussian $i$, $o_i$ is a raw offset predicted per-Gaussian, and $\rho$ is a learnable global scale. The bounded $\tanh$ forces the offset to be a genuine local displacement rather than an absolute position in disguise. Anchors are taken from Hilbert-block centroids and so are exactly reproducible from index alone, which makes them Stage-2 feasible.

4. **Local encoder plus token-local decoder.** I replace the globally-mixed encoder bottleneck with a windowed cross-attention encoder, in which each geometry query attends only to a $\pm\omega$ window of Hilbert blocks. I pair this with a shared per-token decoder MLP of about 1.6M parameters, which replaces the flat $\sim$ 777M-parameter decoder of Can3Tok. The two changes together close almost all of the train-to-test generalisation gap.

5. **Small-scale semantic study.** On 300 scenes, I explore three latent-structure variants and three variants of per-Gaussian InfoNCE supervision (van den Oord et al., 2018). The results motivate the schema-aware Stage 2 design used in the rest of the thesis.

6. **Schema-aware Stage 2 generator.** Because the seven Stage 1 variants produce latents of two structurally different kinds (a single 512-token latent in five of the seven, a separate 16-token layout latent plus a 512-token geometry latent in the other two), Stage 2 has to be schema-aware. I implement and train a hierarchical flow-matching pipeline (Lipman et al., 2023; Peebles and Xie, 2023; Ma et al., 2024) with a single `SceneDiT` for the flat case, a `LayoutDiT` + `GeometryDiT512` pair for the split case, and corresponding unconditional and conditional `CompletionDiT` variants for scene completion. Six Stage 2 jobs covering generation and completion on V5, V6, and V7 are reported in Chapter 4.

7. **Honest characterisation of the residual quality gap.** I identify a residual position-compression issue in the Stage 1 decoder output that propagates to Stage 2, characterise its source, document what I tried to mitigate it, and explain how I would address it in a follow-up. The thesis demonstrates that the architectural directions are correct for the dataset shift from Can3Tok's filtered DL3DV-10K regime to the unfiltered heterogeneous SceneSplat-7K indoor regime; it does not claim photorealistic reconstruction at the end.

8. **Code release.** The full codebase, training scripts, and SLURM job files are available at `https://github.com/Hafeezmhk7/Semantic-Scene-Completion.git`.

## 1.5 Thesis Structure

Chapter 2 places this work in the literature on 3DGS representations, latent generative models, set learning, and semantic representations. Chapter 3 presents the Stage 1 method in detail, going through the data pipeline, the encoder, the reconstruction objective, the decoder, the local encoder change, the small-scale semantic experiments, and the Stage 2 generator design. Chapter 4 reports the empirical findings: the 300-scene exploration first, then the seven-experiment large-scale ablation, then the qualitative reconstructions, the per-Gaussian PCA visualisations, and the Stage 2 results. Chapter 5 reflects on what the results support, on the unfiltered heterogeneous data regime, on the position-compression issue, and on the limitations of the work. Chapter 6 returns to the three research questions. The appendix records the configurations for reproducibility.

Chapter 2

# Related Work

This chapter situates the thesis in four bodies of work. I cover explicit 3D scene representations and where 3DGS sits among them; latent generative modelling of 3D content, with attention to the two-stage VAE+generator recipe; learning on unordered sets and the use of space-filling curves to impose a canonical order; and contrastive plus disentangled representation learning, which informs my small-scale semantic experiments. Each section closes by stating the specific gap the thesis addresses.

## 2.1 3D Scene Representations

**Neural radiance fields.** NeRF (Mildenhall et al., 2020) represents a scene as a continuous volumetric function encoded in the weights of a coordinate-conditioned MLP. It produces photorealistic novel views and remained the dominant approach for several years. Two limitations hold it back as a generative target: rendering is slow because each ray needs hundreds of network evaluations, and the per-scene weight representation has no explicit parameter for a network to predict on a scene level.

**3D Gaussian Splatting.** 3DGS (Kerbl et al., 2023) represents a scene as a set of anisotropic Gaussian primitives rasterised by alpha compositing. The approach renders in real time, matches NeRF on novel-view synthesis, and exposes an explicit per-primitive parameterisation. Each primitive has a position $\mu \in \mathbb{R}^3$, a colour $c \in \mathbb{R}^3$ (the DC spherical-harmonic coefficient, ignoring higher-order view-dependent terms here), an opacity $\alpha \in (0, 1)$, a per-axis scale $s \in \mathbb{R}^3_+$, and a unit quaternion $q$ defining the orientation. The covariance matrix is then $\Sigma = R(q)\,\mathrm{diag}(s^2)\,R(q)^\top$.

Scaffold-GS (Lu et al., 2024) reduces redundancy in a dense 3DGS fit by anchoring per-primitive attributes to a sparser neural scaffold, predicting offsets relative to anchor points rather than absolute positions. This anchor-plus-offset structure is what motivates the position decoding choice in §3.5.2. GaussianCube (Zhang et al., 2024) takes a different route: it fits a fixed Gaussian count at the object level and then transports the primitives onto a regular voxel grid, so a standard convolutional diffusion model can operate on a 3D tensor. Both of these object-level approaches inform my own per-token anchor decoding, but neither extends directly to unbounded indoor scenes.

**Gap.** 3DGS gives an explicit representation, but turning it into a generative target at scene level requires a compact, organised latent. Neither 3DGS itself nor Scaffold-GS provides that latent; building it is the goal of Stage 1 in this thesis.

## 2.2 Latent Generative Modelling of 3D Content

**The two-stage recipe.** A widely used recipe for high-resolution generation is to first train a VAE to compress the signal into a latent (Kingma and Welling, 2014), and then train a generative model in that latent space (Rombach et al., 2022). The generative half has shifted from DDPM (Ho et al., 2020) to flow matching (Lipman et al., 2023; Liu et al., 2023), both typically realised with a transformer backbone such as DiT (Peebles and Xie, 2023) or SiT (Ma et al., 2024). The latent decouples the awkward structure of the raw signal from the easier learning problem of modelling a tame distribution of token sequences.

**Object-level 3D generation.** LION (Zeng et al., 2022) applies hierarchical latent diffusion to point cloud objects. Michelangelo (Zhao et al., 2023) uses a cross-attention Perceiver (Jaegle et al., 2021, 2022) to encode a 3D shape into a fixed set of latent tokens aligned with image and text embeddings; its shape-latent VAE is the architectural ancestor of mine. GaussianCube (Zhang et al., 2024) structures the per-primitive prediction into a position-plus-offset form so a dense convolutional model can operate on it. All three are object-level, with bounded extent and small fixed primitive counts.

**Scene-level 3DGS.** Can3Tok (Gao et al., 2025) is the direct predecessor of this thesis. It uses a Perceiver-style cross-attention encoder to compress a scene's Gaussian primitives into a fixed-size latent, and a transformer-plus-MLP decoder to reconstruct the per-primitive parameters. Can3Tok was trained on the DL3DV-10K dataset (Ling et al., 2024) (around 8,500 training scenes from a 6:1 train/test split), with each scene pre-filtered by a LangSAM-based salient-region selector to retain a clean spatially-coherent subset of 40K Gaussians per scene, and showed text-to-3DGS and image-to-3DGS as downstream generation applications. SceneSplat (Li et al., 2025b) introduces the first generalisable feed-forward 3DGS encoder, pre-trained on its accompanying SceneSplat-7K dataset of 7,916 indoor 3DGS scenes drawn from seven established indoor 3D sources; the dataset comes with per-Gaussian ScanNet-72 (Dai et al., 2017) category labels obtained through vision-language pretraining. SceneSplat-7K is the training corpus throughout this thesis. SceneSplat++ (Li et al., 2025a) extends the dataset and benchmarks language-aligned Gaussian Splatting more thoroughly.

**Gap.** What is required to scale a scene-level 3DGS VAE from Can3Tok's filtered, video-captured DL3DV-10K setting to several thousand unfiltered, heterogeneous indoor scenes coming from seven different capture pipelines, and which architectural choices control whether it converges, has not been systematically studied. The seven-experiment ablation in Chapter 4 is my answer.

## 2.3 Learning on Unordered Sets and Space-Filling Curves

**Permutation-invariant encoders.** PointNet (Qi et al., 2017) established symmetric aggregation as a way to produce permutation-invariant features over point sets. The Perceiver (Jaegle et al., 2021) generalises this by cross-attending a small learned query set against the input, yielding a fixed-size latent that is invariant to input permutation. Permutation invariance is cheap on the encoder side; the difficulty reappears on the decoder side, where any element-wise reconstruction loss compares predicted slot $i$ to target slot $i$ and so implicitly fixes an ordering.

**Canonical orderings via space-filling curves.** Rather than handle permutation in the loss, a complementary approach is to fix a canonical order on the data. Point Transformer V3 (Wu et al., 2024) serialises point cloud tokens along the Hilbert space-filling curve (Skilling, 2004) and shows that this ordering, with its tight locality property (consecutive indices remain spatially adjacent), gives better downstream results than the alternatives such as Z-order (Morton) curves, where high-order bit flips cause long spatial jumps. The same idea, but pushed onto the reconstruction *target*, is the central mechanism behind RQ1 in this thesis.

**Gap.** Canonical serialisation is well known on the processing side of 3D learning but, to the best of my knowledge, has not been used with a fixed-frame coordinate system to make the per-slot 3DGS regression target identifiable for a generative VAE. Doing this in concert with a per-token spatial anchor is what makes the slot-aligned loss learnable in my setup.

## 2.4 Semantic and Disentangled Representations

**Contrastive learning.** Self-supervised contrastive learning (van den Oord et al., 2018; Chen et al., 2020) learns features by pulling together related samples (positives) and pushing apart unrelated ones (negatives) in embedding space. The InfoNCE objective (van den Oord et al., 2018) is the standard tool. I adapt it in two forms in this thesis: a per-Gaussian variant that uses the per-Gaussian ScanNet-72 labels from SceneSplat-7K as the positive criterion, and a scene-level variant that uses the scene's label-distribution histogram.

**Disentangled latent representations.** $\beta$-VAE (Higgins et al., 2017) attempts to disentangle independent factors of variation in the latent by reweighting the KL term in the VAE objective. Locatello et al. (Locatello et al., 2019) show that unsupervised disentanglement is in general unidentifiable without inductive bias, which is why my small-scale experiments rely on structural inductive bias (a layout/geometry split with auxiliary heads and cross-reconstruction) rather than on KL reweighting alone.

**Gap.** Per-Gaussian semantic labels for 3DGS scenes have become available with SceneSplat-7K (Li et al., 2025b). So far they have been used for discriminative scene understanding rather than to add semantic structure to a generative latent. My 300-scene exploration in §4.1 is a first look at how that supervision interacts with a Stage 1 latent designed for downstream generation.

Chapter 3

# Methodology

This chapter walks through the Stage 1 method as it is actually implemented in my code, and then describes the Stage 2 generative design that sits on top of it. The order roughly follows the data path: I start with the problem setup and the variational autoencoder objective (§3.1), then the data pipeline including the Hilbert serialisation (§3.2), the encoder with both its global and local variants (§3.3), the reconstruction objective (§3.4), the decoder including the token-local variant and the anchor-relative position decoding (§3.5), the small-scale semantic experiments that motivated several design choices (§3.7), and finally the Stage 2 generative model (§3.8). I close with an explicit summary of the seven configurations forming the large-scale ablation (§3.9).

The base architecture follows Can3Tok (Gao et al., 2025): a Perceiver-style cross-attention encoder lifting a set of Gaussian primitives into a fixed token sequence, a KL bottleneck, and a transformer-plus-MLP decoder producing back the per-primitive parameters. Everything I add modifies this skeleton, but the encoder topology, the latent dimensionality, the decoder transformer block, and the loss family all come from Can3Tok.

## 3.1 Framework and Problem Setup

**Scene representation.** A scene $\mathcal{G} = \{g_i\}_{i=1}^{N}$ is a set of $N$ Gaussian primitives. Each primitive is a 14-dimensional vector

$$g_i = (\boldsymbol{\mu}_i,\ c_i,\ \alpha_i,\ s_i,\ q_i) \in \mathbb{R}^3 \times \mathbb{R}^3 \times (0,1) \times \mathbb{R}^3_+ \times \mathbb{S}^3, \quad (3.1)$$

with the indices laid out positions [0:3], colour [3:6], opacity [6:7], scale [7:10], quaternion [10:14]. The covariance is $\Sigma_i = R(q_i)\,\mathrm{diag}(s_i^2)\,R(q_i)^\top$.

I work with a fixed budget of $N = 40{,}000$ Gaussians per scene, selected from the larger pre-fitted scene by top-$N$ opacity ranking (the Can3Tok default).

**Autoencoder objective.** Stage 1 is a VAE (Kingma and Welling, 2014) with encoder $E_\phi$ and decoder $D_\theta$. The encoder produces a token-structured latent $Z \in \mathbb{R}^{T\times d}$ with $T = 512$ tokens and $d = 32$ per-token dimensions, drawn from a per-token diagonal Gaussian posterior $q_\phi(Z \mid \mathcal{G}) = \prod_{k=1}^{T} \mathcal{N}(Z_k; \mu_k, \mathrm{diag}(\sigma_k^2))$. In practice the latent is sampled with the standard reparameterisation trick $Z = \mu + \sigma \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$ and $\sigma = \exp(0.5 \log \sigma^2)$. The decoder reconstructs the Gaussian set. The training objective is the standard ELBO,

$$\mathcal{L} = \mathcal{L}_{\text{recon}}(\hat{\mathcal{G}}, \mathcal{G}) \ + \ \beta\, D_{\mathrm{KL}}\big(q_\phi(Z \mid \mathcal{G}) \,\|\, \mathcal{N}(0, I)\big), \quad (3.2)$$

with $\beta = 10^{-5}$ and the KL computed independently per token against an isotropic standard normal on $\mathbb{R}^d$. The very small $\beta$ keeps the latent informative for reconstruction; downstream generation tolerates this because the flow-matching Stage 2 model can warp arbitrary tractable noise into the latent distribution, regardless of how non-Gaussian the latter is. The trade-off is that the resulting latent is less Gaussian than a stronger KL would enforce, which puts more of the modelling burden on the velocity field.

## 3.2 Data Pipeline

### 3.2.1 Dataset

I train on SceneSplat-7K (Li et al., 2025b), a collection of 7,916 indoor 3DGS scenes reconstructed from seven established indoor sources: ScanNet (Dai et al., 2017), ScanNet++, ARKitScenes, Matterport3D, Replica, Hypersim, and 3RScan. The dataset ships with per-Gaussian ScanNet-72 category labels obtained through the SceneSplat vision-language pretraining pipeline, which I use for the semantic experiments in §3.7.

The main training set in the large-scale runs is 3,800 chunked indoor scenes preprocessed at $8 \times 8$ tile stride with $6 \times 6$ overlap from the ScanNet/Matterport sources. Three additional sources are folded in for the actual runs: 800 full indoor scenes, 1,290 ARKitScenes, and 906 ScanNet++ v2 scenes. Per-Gaussian semantic labels are disabled for the additional sources to avoid cross-dataset taxonomy mismatch. The total training pool is about 6,796 scenes (the remainder of SceneSplat-7K is reserved for held-out validation and for sources not used in this work). Validation uses 100 held-out full scenes (the primary thesis-target metric) and 88 disjoint held-out chunks (an in-distribution diagnostic).

Each scene is normalised into a canonical sphere of radius 10 m. For chunked scenes, a global coordinate frame is precomputed from the union of all chunks belonging to the same parent scene and stored in a `norm_factor.npy` file, so every chunk shares one consistent frame. Full scenes are normalised per scene. This normalisation is the precondition for any kind of cross-scene canonical ordering.

### 3.2.2 Hilbert Serialisation

After the per-scene opacity-based sampling of $N = 40{,}000$ Gaussians, I serialise the selected primitives along a 3D Hilbert curve (Skilling, 2004). Formally, the serialisation is a map $f : \mathbb{R}^3 \to \mathbb{Z}_{\geq 0}$ defined as the composition $f = \mathcal{H} \circ q$ of an affine grid quantisation and a Hilbert encoding,

$$q : \mathbb{R}^3 \to [0, 2^B)^3, \qquad q(p) \ = \ \left\lfloor \frac{p + R}{2R} \cdot (2^B - 1) \right\rfloor, \quad (3.3)$$

$$\mathcal{H} : [0, 2^B)^3 \ \to \ \{0, \ldots, 2^{3B} - 1\}, \quad (3.4)$$

where $R = 10$ m is the canonical normalisation radius, $B = 10$ bits per axis (a $1024^3$ grid), and $\mathcal{H}$ is the Skilling AxesToTranspose construction, which interleaves the bits of the grid coordinates along the Hilbert traversal order. The Hilbert key of Gaus-

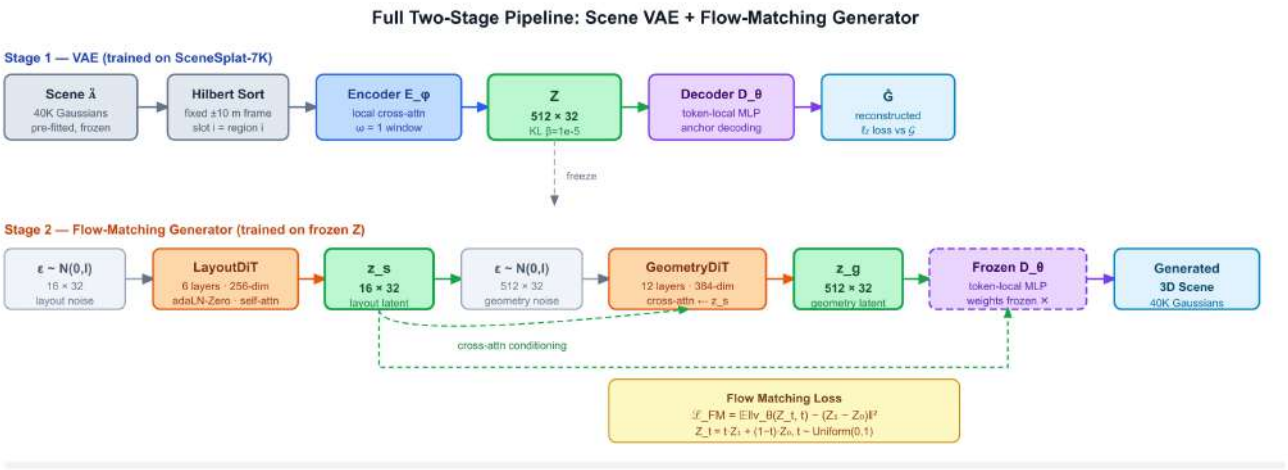



Figure 3.1: **Stage 1 plus Stage 2 pipeline overview.** A raw 3DGS scene is normalised into the canonical $[-10, 10]$ m frame and serialised along a fixed-frame Hilbert curve. The local cross-attention encoder produces a structured latent $Z \in \mathbb{R}^{512\times32}$. The token-local MLP decoder reconstructs the scene using anchor-relative position decoding. The frozen Stage 1 latent is then the training target of a hierarchical flow-matching Stage 2 generator. Dashed border marks the frozen decoder reused during Stage 2 inference.

sian $i$ is then $h(i) = f(p_i)$, and tokens are sorted by $h(i)$ ascending. The key satisfies the locality property that if $|h(i) - h(j)|$ is small, then $\|p_i - p_j\|_\infty$ is also small; the converse does not hold in general, but no pair of consecutive indices crosses the grid (which is what the alternative Z-order curve does not guarantee). The flag in the code is called `morton_order`, but the actual curve is selected by `order_curve` and is `hilbert` in every run reported.

Two design points are critical here. First, the locality property of the Hilbert curve: consecutive indices are always spatially adjacent in the underlying grid, with no long jumps. The alternative Z-order (Morton) curve fails this property because a high-order bit flip moves the curve to a distant cell, which is the precise reason Wu et al. (2024) prefer the Hilbert order for point cloud processing. Second, the frame is *fixed*, not per-scene. With a per-scene frame the curve would traverse the same relative grid in every scene, but the same absolute slot index would correspond to a different physical region across scenes, defeating the purpose of the ordering for the slot-aligned loss. With a fixed canonical frame, slot $i$ corresponds to roughly the same physical region across all scenes, which makes the reconstruction target identifiable in the way RQ1 requires.

After sorting, contiguous blocks of $g = \lceil N/T \rceil = \lceil 40000/512 \rceil$ 79 Gaussians correspond to compact local clusters. The Hilbert order plus this block partition has a direct empirical signature on the input: the mean consecutive-slot distance drops from $0.06$ m under opacity ranking to $0.03$ m under Hilbert ordering, both measured on the same held-out scene shown in Figure 3.3. The block partition is reused in two places downstream: the per-token Hilbert-block anchors used by the position scaffold (§3.5.2), and the per-token decoding done by the token-local decoder (§3.5). Token $k$ owns block $k$ throughout.

### 3.2.3 Colour Decomposition

Colour is decomposed into a per-scene mean and a per-primitive residual. The mean colour $\bar{c} \in \mathbb{R}^3$ is predicted from the scene-level shape embedding (or, in the disentangled variants, from $z_s$) by a small `MeanColorHead`, and the decoder produces a residual $r_i \in \mathbb{R}^3$ for each primitive. The reconstructed colour is then

$$\hat{c}_i = \hat{\bar{c}} + \hat{r}_i. \tag{3.5}$$

This DC/AC split (an analogy to DC and AC coefficients in spectral decompositions) is on in every large-scale run. Its purpose is to remove the easy global colour-palette signal from the per-primitive prediction so the decoder can use its capacity on the harder local colour variation. In practice the per-scene mean is well predicted from the global summary, and the residual head only has to model local deviations from that mean.

## 3.3 Encoder

The encoder topology comes from Can3Tok (Gao et al., 2025) and the underlying Perceiver design (Jaegle et al., 2021, 2022). I describe the global form first, then the local variant that is one of the central architectural changes of the thesis.

**Input features.** Each of the $N$ Gaussians produces an 18-dimensional raw feature vector at the input: voxel-centre coordinates from an FNV hash of a $40^3$ grid (3 dims), a voxel id (1 dim), the normalised position (3 dims), the colour (3 dims), opacity (1 dim), scale (3 dims), and quaternion (4 dims). The position and voxel centre are each lifted to 8-frequency Fourier features (Tancik et al., 2020), expanding those six raw position dimensions into $6 \times (2 \times 8) = 96$ Fourier dimensions before the rest of the raw features are concatenated. The full per-Gaussian feature vector is then projected to the model width $w = 384$ by a single linear input layer.

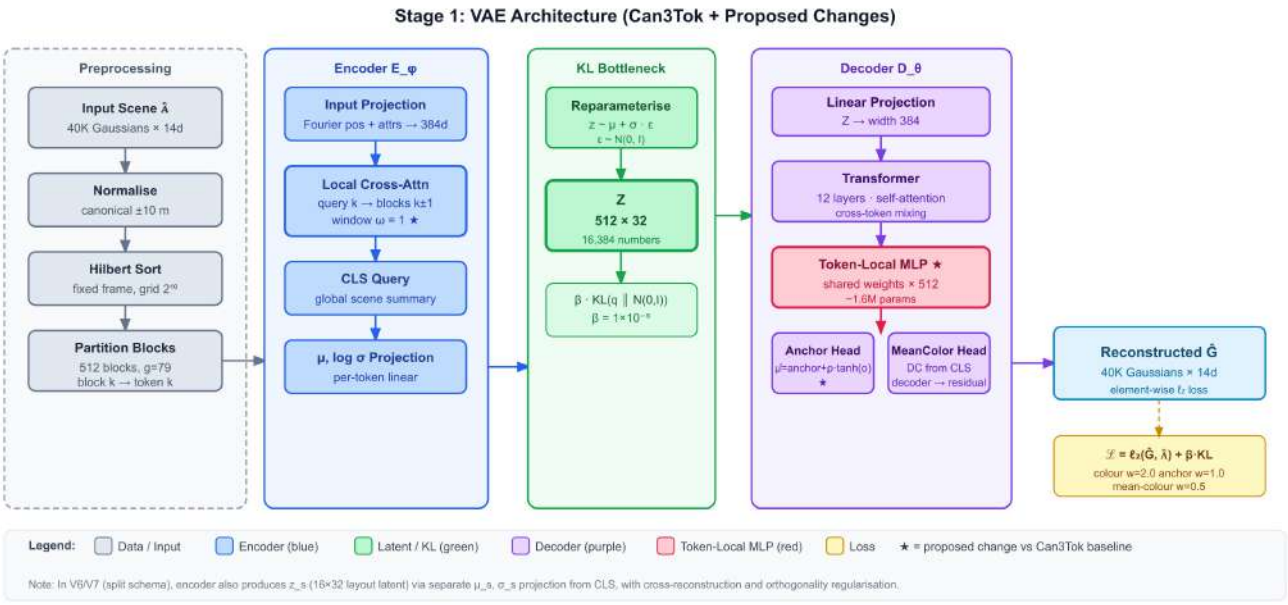



Figure 3.2: **Stage 1 VAE architecture.** Detailed view of the encoder, KL bottleneck, and decoder. The input scene is normalised, Hilbert-sorted, and partitioned into 512 blocks of 79 Gaussians each. The local cross-attention encoder ($\omega$=1, ★) produces per-token posteriors; the decoder uses a token-local shared MLP (★) with anchor-relative position decoding and a `MeanColorHead` for the DC colour component. ★ marks the two proposed changes versus the Can3Tok baseline.

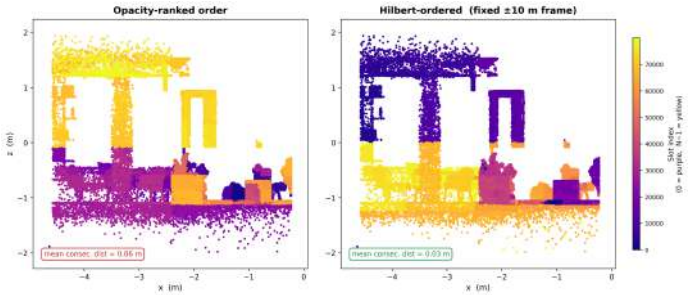



Figure 3.3: **Slot ordering stability.** Top-down scatter of 80,000 Gaussians from a held-out validation scene, coloured by array index from purple (slot 0) to yellow (slot $N-1$). *Left:* opacity-ranked order: consecutive slots correspond to different spatial regions (the ceiling receives high-opacity slots, the floor low-opacity slots), so slot $i$ maps to a different physical location in every scene. *Right:* Hilbert-ordered in a fixed $\pm 10$ m canonical frame: colour sweeps smoothly across the room and consecutive slots are always spatial neighbours (mean consecutive distance 0.03 m versus 0.06 m with opacity ordering). This spatial stability is what makes the slot-aligned $\ell_2$ reconstruction target learnable across scenes.

**Global cross-attention bottleneck.** The Perceiver block uses a set of $1 + K$ learnable query tokens that cross-attend to the projected per-Gaussian features. Query 0 produces the scene-level shape embedding $e \in \mathbb{R}^w$; the remaining $K$ tokens produce per-region geometry tokens. The token count $K$ is set by the YAML config (the value used in this work yields $T = 512$ latent tokens after the KL projection). Six self-attention layers then mix the query tokens. The output is a $(1 + K) \times w$ token sequence that is permutation-invariant in the input Gaussians.

**KL bottleneck.** A linear projection per token produces the per-token mean and log-variance, the latent is sampled by reparameterisation, and an inverse linear projection brings it back to width $w$ for the decoder. The pre-KL flat embed dimension and the per-token dimension are constrained so that the total number of latent numbers is $T \cdot d = 512 \cdot 32 = 16384$. This dimensionality is held fixed across all seven large-scale variants; only its *structure* changes, not its budget.

**Spatially local encoder.** The single biggest change to the encoder is a windowed local variant. Each of the $K = 512$ geometry queries attends only to the Gaussians belonging to a local Hilbert window of $\pm\omega$ blocks around its own block, with $\omega = 1$ in my runs (so a query sees three contiguous Hilbert blocks). A single global CLS query still attends to all $N$ Gaussians, so the scene-level shape embedding remains a global summary. After the cross-attention, the encoder runs no global self-attention; cross-token mixing is handled later in the decoder transformer.

Two consequences follow. First, every latent token is the encoding of a local spatial cluster, not a global mixture of the whole scene. Two scenes with a similar local patch (a flat floor, a uniform wall) activate the same local features. This is the compositional structure I show in Chapter 4 to be the key to closing the train-to-test generalisation gap. Second, when paired with `structured_latent` (which uses the per-token posterior directly, instead of flattening and remixing it through a single dense projection), the latent now has the property latent token $k$ = encoder block $k$ = input Hilbert block $k$, end to end. The mapping is exact, not approximate.

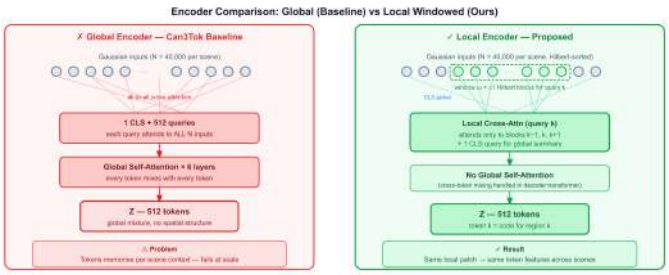



Figure 3.4: **Global vs. local encoder.** *Left:* the global cross-attention encoder gives every geometry query a chance to attend to all $N$ Gaussians, with six layers of global self-attention mixing all tokens. *Right:* the local encoder restricts each geometry query to a $\pm\omega$ Hilbert window ($\omega$=1); only the single CLS query attends globally. The absence of global self-attention produces compositional, spatially structured tokens that generalise across scenes.

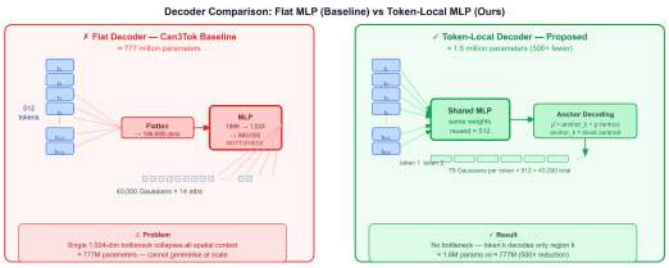



Figure 3.5: **Flat vs. token-local decoder.** *Left:* the flat decoder (Can3Tok baseline) concatenates all 512 token vectors and passes them through a single 1,024-dim MLP bottleneck (≈777M params). *Right:* the token-local decoder applies a shared MLP independently to each of the 512 tokens, with anchor-relative position decoding (≈1.6M params, 500× fewer). Token $k$ decodes only the 79 Gaussians belonging to its Hilbert block.

## 3.4 Reconstruction Objective

The reconstruction term in Eq. (3.2) is an element-wise $\ell_2$ residual on the 14-dimensional Gaussian vector. With the Hilbert order in place and the per-token block partition fixed, the slot-aligned loss has a stable target.

In the no-scaffold variants the predicted and target are compared directly. Per scene, the reconstruction term is

$$\mathcal{L}_{\text{recon}} \;=\; \frac{1}{N}\sum_{i=1}^{N}\big\|\hat{g}_i - g_i\big\|_2, \tag{3.6}$$

and is then averaged over the batch. The sum is implicitly broken into the per-attribute components (position, colour, opacity, scale, rotation) for reporting. A colour weight of 2.0 and a separate scalar weight on the mean-colour-prediction MSE are used in every large-scale run.

In the scaffolded variants, position is decoded relative to a per-token anchor (§3.5.2), so the position part of the loss is computed on the offset $\hat{o}_i = \hat{\boldsymbol{\mu}}_i - a_{\tau(i)}$ rather than on the absolute coordinate. An additional anchor loss supervises the per-token anchor to be the centroid of its Hilbert block,

$$\mathcal{L}_{\text{anchor}} \;=\; \frac{1}{T}\sum_{k=1}^{T}\big\|\hat{a}_k - \bar{\boldsymbol{\mu}}_{\text{block}(k)}\big\|_2^2, \tag{3.7}$$

where $\bar{\boldsymbol{\mu}}_{\text{block}(k)} = \frac{1}{g}\sum_{i\in\text{block}(k)}\boldsymbol{\mu}_i$ is the mean position of the Gaussians assigned to block $k$ by the Hilbert order. This is what makes the anchor head a meaningful localiser and not just a pass-through.

## 3.5 Decoder

The decoder is a 12-layer transformer at width $w = 384$ followed by a per-primitive head that produces the 14-dimensional output. I describe the per-primitive head in two variants below, and then the anchor-relative position decoding.

### 3.5.1 Flat vs. Token-Local Decoder

The Can3Tok decoder ends with a single flat MLP. Concretely it concatenates all $T \cdot w = 512 \cdot 384 = 196{,}608$ post-transformer features into one vector, projects through a 1024-dimensional hidden bottleneck (an architectural hyperparameter of Can3Tok, fixed here for fair comparison), and then projects to $N \cdot 14 = 560{,}000$ outputs. The parameter count is dominated by these two linear layers: $196{,}608 \times 1024 \approx 201$M and $1024 \times 560{,}000 \approx 573$M, for a total of about 777M parameters at $N = 40{,}000$.

The 1024-dimensional bottleneck is the choke point. Every scene-specific local detail has to flow through it. With a few hundred scenes the network can almost memorise the relevant codes, but with thousands of scenes that memorisation strategy breaks and the model produces a generic, washed-out scene. The bottleneck cannot store the per-scene context needed to disambiguate similar input patches across the training set.

The token-local decoder replaces the flat MLP with a shared per-token MLP applied independently to each of the $T = 512$ decoder tokens. Each token decodes its own $g = 79$ Gaussians:

$$\hat{g}_{[k]} \;=\; \text{MLP}_\theta\big(h_k\big), \quad k = 1, \ldots, T, \tag{3.8}$$

where $h_k \in \mathbb{R}^w$ is the post-transformer feature of token $k$ and $\hat{g}_{[k]} \in \mathbb{R}^{g\times 14}$ is the predicted block. The MLP weights are shared across all tokens. The total parameter count drops from about 777M to about 1.6M, while the effective representation bandwidth per token rises from $1024/T \approx 2$ to $w = 384$. There is no longer a cross-scene bottleneck on the per-Gaussian path.

Token-local decoding gives a clean spatial structure: token $k$ attends to its local context through the (windowed) encoder, the latent slot $k$ is the per-token posterior of that context, the transformer block $k$ mixes it with its neighbours, the position scaffold places token $k$ at the centroid of its Hilbert block, and the per-token MLP decodes the $g$ Gaussians within. Each step matches the others. This end-to-end consistency is what makes the architecture generalise.

### 3.5.2 Anchor-Relative Position Decoding

In the baseline decoder, the position head predicts the absolute coordinate of every Gaussian. Even with a Hilbert-ordered input,

this still requires the network to learn an absolute coordinate target across scenes. Inspired by Scaffold-GS (Lu et al., 2024) and the structured prediction in GaussianCube (Zhang et al., 2024), I split the position prediction into an anchor and a bounded local offset,

$$\hat{\boldsymbol{\mu}}_i \;=\; a_{\tau(i)} \;+\; \rho\,\tanh(o_i), \tag{3.9}$$

where $a_{\tau(i)} \in \mathbb{R}^3$ is the anchor of the token owning Gaussian $i$, $o_i \in \mathbb{R}^3$ is a raw offset predicted by the per-Gaussian decoder, and $\rho > 0$ is a single learnable global scalar shared across all tokens (exp-parameterised, initialised at 2.0). The anchors are predicted by a small head from the post-transformer token features and supervised by Eq. (3.7) towards the Hilbert-block centroids.

Two practical points are easy to miss in the implementation but matter for downstream Stage 2:

- **Hilbert-block assignment.** The token that owns Gaussian $i$ is determined by the Hilbert order, not by voxel containment. Concretely, $\tau(i) = \lfloor i/g \rfloor$ with $g = \lceil N/T \rceil = 79$. This is reproducible at inference time from the index alone, with no need for the ground-truth position; it is the property that makes the scaffold Stage-2 feasible.
- **Bounded local offset.** The $\tanh$ bounding is what makes the anchor genuinely informative. Without it, the offset is unbounded and the network can simply offset away from a meaningless anchor, falling back to absolute position prediction. With the $\tanh$, the local-region structure becomes the dominant signal.

Because $\rho$ is a single global scalar shared across all tokens, training pressure can push it to a small value, in which case every per-token offset is bounded by the same conservative range and the decoder's output extent narrows. I return to the empirical consequences of this design choice in §5.3.

## 3.6 Disentanglement and the Two Latent Schemas

Two of the seven Stage 1 variants additionally split the latent into a global layout component and a local geometry component, motivated by the small-scale results in §3.7 and detailed in the local-disentangle code path.

In these variants, a separate global layout latent $z_s \in \mathbb{R}^{16\times32}$ is produced by a small projection of the encoder CLS into mean and log-variance vectors; this $z_s$ is KL-regularised on its own. The geometry latent is the structured per-token $z_g \in \mathbb{R}^{512\times32}$ coming from the local encoder. The total per-scene latent has $16 + 512 = 528$ tokens, but $z_s$ is consumed as a separate cross-attention context by the decoder rather than concatenated to the main token sequence; the geometry budget is unchanged.

Two structural losses help bind the meaning of the split. The first is a cross-reconstruction term: for a batch of $B$ scenes, let $\pi$ be a random derangement on $\{1, \dots, B\}$ (so $\pi(b) \neq b$ for all $b$). I swap each scene's layout latent with that of its derangement partner and re-decode against the original geometry:

$$\mathcal{L}_{\text{cross}} \;=\; \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}_{\text{recon}}\big(D_\theta(z_g^{(b)}, z_s^{(\pi(b))}),\, \mathcal{G}^{(b)}\big). \tag{3.10}$$

Because the geometry is graded against the original scene under the swapped layout, $z_s$ is pressured not to carry geometry. The second is an orthogonality term that decorrelates random dimensional subsets of the layout and geometry means within a batch,

$$\mathcal{L}_{\text{ortho}} \;=\; \big\|\bar{\mu}_s^{\top}\bar{\mu}_g\big\|_F^2, \tag{3.11}$$

where $\bar{\mu}_s$ and $\bar{\mu}_g$ are batch-centred slices of the layout and geometry means, discouraging trivial duplication between the two subspaces.

Three small auxiliary heads on $z_s$ predict the scene's mean colour, ScanNet-72 label distribution, and per-category spatial centroids, all of which are unambiguously layout-level quantities.

For the rest of the document I call the latent schema of the seven variants in two groups:

- *Flat schema* (variants 1–5): a single 512-token latent. The decoder consumes it directly; there is no separate $z_s$.
- *Split schema* (variants 6 and 7): a 16-token $z_s$ plus a 512-token $z_g$.

Stage 2 has to react to this difference.

### 3.6.1 Total Stage 1 Objective

Collecting the terms introduced above, the full Stage 1 training objective is a weighted sum. For variant V6, which has every term active, the objective is

$$\begin{aligned}\mathcal{L}_{\text{Stage 1}} \;=\; &\mathcal{L}_{\text{recon}} \;+\; \beta\,\mathcal{L}_{\text{KL}} \;+\; \lambda_{\text{mc}}\,\mathcal{L}_{\text{meancolor}} \;+\; \lambda_{\text{anchor}}\,\mathcal{L}_{\text{anchor}} \\ &+\; \lambda_{\text{ssem}}\,\mathcal{L}_{\text{scenesem}} \;+\; \lambda_{\text{layout}}\,\mathcal{L}_{\text{layout}} \;+\; \lambda_{\text{cross}}\,\mathcal{L}_{\text{cross}} \\ &+\; \lambda_{\text{ortho}}\,\mathcal{L}_{\text{ortho}} \;+\; \lambda_{\text{NCE}}\,\mathcal{L}_{\text{NCE}},\end{aligned} \tag{3.12}$$

with $\beta = 10^{-5}$, $\lambda_{\text{mc}} = 0.5$, $\lambda_{\text{anchor}} = 1.0$, $\lambda_{\text{ssem}} = 0.3$, $\lambda_{\text{layout}} = 0.3$, $\lambda_{\text{cross}} = 0.3$, $\lambda_{\text{ortho}} = 0.1$, and $\lambda_{\text{NCE}} = 0.4$. The colour upweight of 2.0 is applied *inside* $\mathcal{L}_{\text{recon}}$ in Eq. (3.6), so it weights only the colour-channel term of the per-Gaussian norm and does not appear as a separate $\lambda$ in Eq. (3.12).

The other six variants use the same template with subsets of the optional terms gated by the corresponding switches in Table 3.1. Concretely, $\mathcal{L}_{\text{anchor}}$ is active only when the position scaffold is on (V3 and V6); $\mathcal{L}_{\text{ssem}}$, $\mathcal{L}_{\text{layout}}$, $\mathcal{L}_{\text{cross}}$, $\mathcal{L}_{\text{ortho}}$, and $\mathcal{L}_{\text{NCE}}$ are active only when the local disentangle plus per-Gaussian InfoNCE switches are on (V6 and V7); and the remaining terms ($\mathcal{L}_{\text{recon}}$, $\mathcal{L}_{\text{KL}}$, $\lambda_{\text{mc}}\,\mathcal{L}_{\text{meancolor}}$) are active in every variant.

## 3.7 Small-Scale Semantic Experiments

Before committing to a Stage 1 design at scale, I ran a series of 300-scene experiments to understand which latent-structure choices were even feasible and which would survive the move to thousands of scenes. The motivation for the disentanglement step in §3.6 comes directly from these.

**Latent disentanglement variants.** Three configurations were tested.

**Strategy C (baseline)** No disentanglement. The latent is a single undifferentiated 512-token sequence.

**Strategy A** A disentangled split where $z_s$ is derived from the shape embedding via a separate mean/variance projection and $z_g$ comes from the geometry queries. Auxiliary heads on $z_s$ predict mean colour, label distribution, and per-category centroids. Cross-reconstruction and orthogonality are active.

**Strategy B1** A different conditioning route: a deterministic `Layout16Projector` produces a fixed 16-token layout code from the shape embedding (no VAE sampling on $z_s$). The latent has 512 pure geometry tokens, and the layout code conditions every decoder layer through cross-attention. Because the layout code is deterministic, the same partial scene always produces the same layout, which makes B1 ideal for the completion task.

A fourth variant, an additive-bias version of B (Strategy B2), was also tried; it failed to disentangle (the additive bias is absorbed by the decoder as a constant and provides no useful conditioning). I report it briefly in Chapter 4 for completeness.

**Per-Gaussian InfoNCE.** On top of the disentanglement variants, a projection head maps each decoded Gaussian to a 32-d L2-normalised embedding $f_i$. Following van den Oord et al. (2018), the loss pulls together Gaussians that share a ScanNet-72 label and pushes apart Gaussians from different categories. With a temperature $\tau$,

$$\mathcal{L}_{\text{NCE}} \;=\; -\frac{1}{M}\sum_{i=1}^{M}\log\frac{\exp(f_i^\top p_{y_i}/\tau)}{\sum_{k=1}^{K_{\text{cats}}}\exp(f_i^\top p_k/\tau)}, \tag{3.13}$$

where $p_k$ is the L2-normalised mean feature of category $k$ computed over the current batch, $y_i$ is the Gaussian's ScanNet-72 label, $M$ is the number of labelled Gaussians in the batch, and $K_{\text{cats}}$ is the number of distinct categories that appear in the batch (computed on the fly via `torch.unique`; only categories actually present contribute a prototype). I use $\tau = 0.07$ in the large-scale V6 and V7 runs reported in Chapter 4 (the temperature used by the original ScanNet72 contrastive baseline I adapted), and $\tau = 0.10$ in some of the small-scale exploratory runs of §4.1. Three flavours were tested at small scale: pgNCE on per-Gaussian decoder features, PoolNCE on a permutation-invariant pool of $z_s$ features against a scene-level positive, and Pool+pgNCE combining both. Of these, Pool+pgNCE gave the cleanest cross-scene swap behaviour. None of the three changes the reconstruction loss materially; the supervision is a soft secondary signal.

**What carried over to the large-scale runs.** The 300-scene results (Chapter 4, §4.1) confirmed that a disentangled latent with auxiliary heads is feasible, that B1's deterministic layout code is the right structure for partial-scan completion, and that per-Gaussian semantic features cluster meaningfully. The large-scale runs include two configurations with this disentanglement plus per-Gaussian InfoNCE (variants 6 and 7); five baseline-style variants without it complete the ablation. Strategy B2 was not carried over.

## 3.8 Stage 2 Generative Model

The second stage models the distribution of the frozen Stage 1 latent with flow matching (Lipman et al., 2023; Liu et al., 2023). I use the linear rectified path: given clean latent $Z_1$ and Gaussian noise $Z_0 \sim \mathcal{N}(0, I)$, the interpolated state is

$$Z_t \;=\; t\,Z_1 \;+\; (1-t)\,Z_0, \qquad t \in [0,1]. \tag{3.14}$$

Differentiating with respect to $t$ gives a constant velocity target $\dot{Z}_t = Z_1 - Z_0$, so a velocity network $v_\theta(Z_t, t)$ that matches this target can be integrated from $Z_0$ to recover $Z_1$. The training loss is

$$\mathcal{L}_{\text{FM}} \;=\; \mathbb{E}_{t,Z_1,Z_0}\Big[\,\big\|v_\theta(Z_t,t) - (Z_1 - Z_0)\big\|_2^2\,\Big], \tag{3.15}$$

with $t$ drawn uniformly on the closed unit interval. At inference, the velocity field is integrated by a forward Euler step from $Z_0$ to recover a sample of $Z_1$.

The backbone is a Diffusion Transformer (DiT) (Peebles and Xie, 2023; Ma et al., 2024) with adaLN-Zero timestep conditioning. RoPE (Su et al., 2021) is supported for the geometry path as an ablation against learned absolute positional embeddings.

**Schema-aware Stage 2 design.** Because the seven Stage 1 variants split into a flat and a split schema, Stage 2 has to read the schema from the checkpoint and route correctly. There are four model classes, instantiated according to which Stage 1 was used.

- *Flat schema, generation.* A single `SceneDiT` operates over the 512-token latent. Self-attention only (no cross-attention), adaLN-Zero on the timestep. This is essentially `LayoutDiT` widened from 16 to 512 tokens.
- *Flat schema, mean colour.* For colour-residual checkpoints, the per-scene mean colour is not part of the latent (it is read from the shape embedding in Stage 1). I train a small `DCHead` that models a conditional Gaussian $q(\bar{c} \mid Z)$ from a permutation-invariant pool of $Z$, so the missing DC component can be recovered at generation time.
- *Split schema, generation.* Two stages: a small `LayoutDiT` generates $z_s$ from noise, and a deeper `GeometryDiT512` generates $z_g$ conditioned on $z_s$ by cross-attention at every block. The frozen Stage 1 decoder then produces the Gaussians.
- *Completion.* For the split schema, `CompletionDiT` conditions on $z_s$ via cross-attention while masking a random fraction of $z_g$ tokens (30–80% coverage) and denoising the masked positions. For the flat structured case (variant 5), an unconditional variant `CompletionDiTUncond` performs the same masked denoising with self-attention only.

Completion is only meaningful when token $k$ corresponds to a spatial region, which holds for variants 5, 6, and 7. For the global-flat variants 1–4 a token is a global mixture and masking it has no spatial meaning; I skip completion for those.

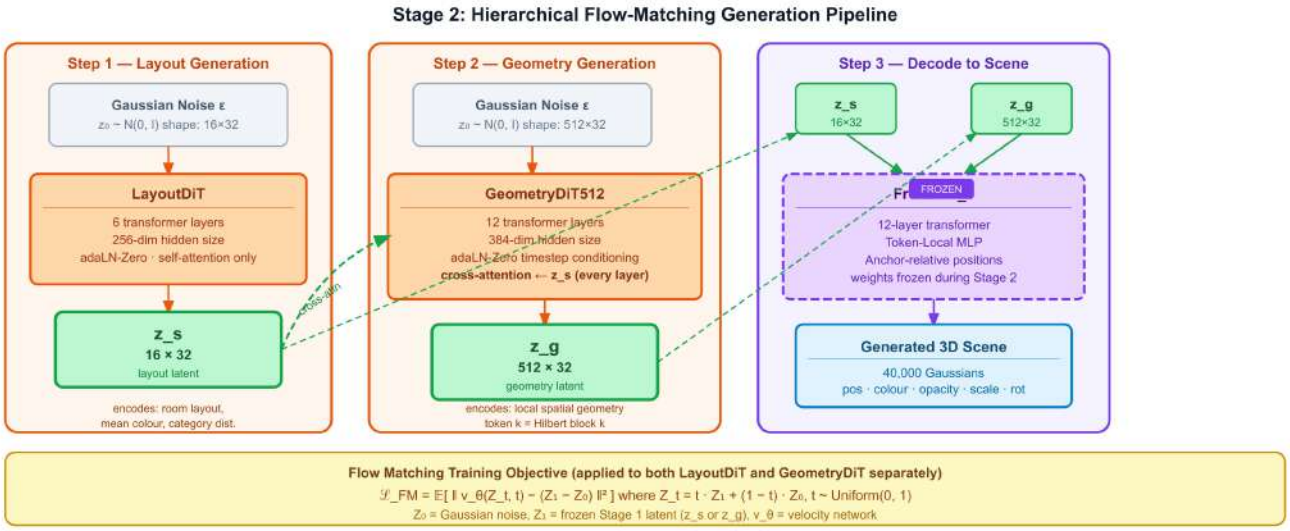



Figure 3.6: **Stage 2 hierarchical generation pipeline (split schema).** Step 1: `LayoutDiT` (6 layers, 256-dim) integrates a rectified flow from noise to produce the layout latent $z_s \in \mathbb{R}^{16\times32}$. Step 2: `GeometryDiT512` (12 layers, 384-dim) generates the geometry latent $z_g \in \mathbb{R}^{512\times32}$, conditioned on $z_s$ via cross-attention at every layer. Step 3: the frozen Stage 1 decoder converts $z_s$ and $z_g$ back into a full 40K-Gaussian scene. Both DiTs are trained with the rectified flow matching loss shown. Dashed border marks the frozen decoder.

## 3.9 Configuration Summary for the Seven Experiments

All Stage 1 runs share the architecture defaults: transformer width $w = 384$, 12 attention heads, 6 encoder layers, 12 decoder layers, $T = 512$ tokens, $d = 32$ per token, 8-frequency Fourier features. Training uses AdamW (Loshchilov and Hutter, 2019) with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $10^{-2}$, base learning rate $10^{-4}$, cosine decay with warm restarts at period $T_0 = 800$ epochs, and 300-step linear warmup. The KL weight is $\beta = 10^{-5}$, and training is in bf16 mixed precision on 2×H100 GPUs (Snellius project EINF-16254). Batch size is 90 for the runs without local encoder (variants 1–4) and 64 for the runs with local encoder (variants 5–7). Validation runs every 10 epochs.

Table 3.1 summarises which switches are active in each of the seven variants. The table is the single most useful map of the ablation.

Three observations are worth making explicit. First, the only difference between V6 and V7 is the position scaffold, which gives a clean A/B test of what the anchor decomposition contributes once everything else (Hilbert order, local encoder, token-local decoder, disentanglement, per-Gaussian InfoNCE) is in place. Second, V1, V4, V5 share opacity-rank input order (no Hilbert reorder), so they are not strictly comparable to the reordered variants on the position component of the loss; they are nonetheless useful for isolating the decoder change (V1 vs. V4) and the encoder change (V4 vs. V5) under controlled conditions. Third, the parameter count difference between the flat decoder (about 777M) and the token-local decoder (about 1.6M) is roughly two orders of magnitude, so any comparison between V1–V3 and V4–V7 mixes a capacity change with an architectural change. I revisit both observations in Chapter 5.

Table 3.1: **Configuration matrix for the seven Stage 1 variants.** A check mark indicates the flag is active in the run, a dash that it is not. Variants are ordered so that each adds one (or sometimes two) switches on top of the baseline. Variants 6 and 7 differ only in whether the position scaffold is on. Variants 1–5 are flat schema (single 512-token latent); variants 6 and 7 are split schema (16-token $z_s$ plus 512-token $z_g$). The colour-residual decomposition and the KL bottleneck are on in every variant. The bottom group records the latent schema, the decoder parameter count, and the Stage 2 tasks the variant supports.

| | V1 | V2 | V3 | V4 | V5 | V6 | V7 |
|---|---|---|---|---|---|---|---|
| Hilbert reorder | – | ✓ | ✓ | – | – | ✓ | ✓ |
| Position scaffold (Hilbert-block) | – | – | ✓ | – | – | ✓ | – |
| Anchor-relative decoding | – | – | ✓ | – | – | ✓ | – |
| Token-local decoder | – | – | – | ✓ | ✓ | ✓ | ✓ |
| Local encoder ($\omega = 1$) | – | – | – | – | ✓ | ✓ | ✓ |
| Structured per-token latent | – | – | – | – | ✓ | ✓ | ✓ |
| Local disentangle ($z_s$ + $z_g$) | – | – | – | – | – | ✓ | ✓ |
| Cross-recon + ortho | – | – | – | – | – | ✓ | ✓ |
| Scene-semantic head | – | – | – | – | – | ✓ | ✓ |
| Scene-layout head | – | – | – | – | – | ✓ | ✓ |
| Per-Gaussian InfoNCE | – | – | – | – | – | ✓ | ✓ |
| Colour residual (DC/AC split) | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| *Latent schema* | Flat | Flat | Flat | Flat | Flat | Split | Split |
| *Decoder params* | ∼777M | ∼777M | ∼777M | ∼1.6M | ∼1.6M | ∼1.6M | ∼1.6M |
| *Stage 2 tasks* | Comp. | Comp. | Comp. | Comp. | Comp. | Comp. + Gen. | Comp. + Gen. |

Chapter 4

# Results

This chapter reports the empirical findings. I begin with the 300-scene exploration that motivated the disentangled latent and the per-Gaussian semantic supervision (§4.1), then move to the seven-variant large-scale ablation that is the main quantitative contribution (§4.2). Per-component loss breakdowns and validation convergence are reported in §4.3 and §4.4. Qualitative Stage 1 reconstructions and the per-Gaussian PCA visualisations are in §4.5 and §4.6. Stage 2 results are in §4.7.

Throughout this chapter the validation metric for Stage 1 is the same element-wise $\ell_2$ residual on the 14-dim Gaussian vector that is used in training (Eq. (3.6)). Two validation sets are reported, the 100 held-out full scenes ($\text{val}^{\text{full}}$, the primary thesis-target metric) and the 88 held-out chunks ($\text{val}^{\text{chunk}}$, an in-distribution diagnostic). For Stage 2 the metric is the rectified-flow velocity MSE (Eq. (3.15)) on the corresponding latent. All numbers are read directly from the SLURM job logs.

## 4.1 300-Scene Exploratory Study

**Setup.** I subsampled 300 scenes from the chunked ScanNet split (180 train, 60 val, 60 held-out test for swap diagnostics) and ran a sweep over the three latent-structure strategies described in §3.7 and the three InfoNCE flavours. All other hyperparameters match the eventual large-scale configuration, except for batch size 16, 200 epochs, and the validation set being a 60-scene subset of the chunked split rather than the 100 held-out full scenes used at large scale.

**Reconstruction is stable across variants.** At 300 scenes the network has enough capacity to fit any of the four variants, so the raw reconstruction quality is similar across the sweep. The validation $\ell_2$ at epoch 200 sits in the 35–40 range across all four variants. This is consistent with the failure mode at scale being one of generalisation, not of representational capacity.

**Layout-vs-geometry stability.** A more diagnostic metric is the *stability* of $z_s$ under partial-scan completion, defined as the cosine similarity between $z_s$ produced from a full scene and $z_s$ from a 40%-randomly-masked partial version. A high value (close to 1) indicates the layout component is stable under missing data, which is the property the disentanglement is meant to enforce. Table 4.1 gives the comparison.

Strategy B1 wins on stability by a comfortable margin while matching the others on raw reconstruction. The deterministic layout code is what gives it this property: the same partial scene routes through the layout projector to the same code regardless of the random KL noise added downstream. Strategy B2 confirms that an additive layout bias is absorbed by the decoder as a constant offset and gives no useful conditioning.

Table 4.1: **Layout stability at 300 scenes.** Higher Stab.@40% is better. Strategy B1 is the deterministic-layout variant and gives the most stable $z_s$ under masking; Strategy A is the VAE-sampled variant; Strategy B2 (additive bias) failed to disentangle and is shown for completeness; Strategy C (no disentanglement) has no $z_s$ to measure.

| | Val. recon. $\ell_2$ (↓) | Stab.@40% (↑) |
|---|---|---|
| Strategy A | 36.1 | 0.62 |
| Strategy B1 | 35.9 | **0.76** |
| Strategy B2 | 36.4 | 0.31 |
| Strategy C | 35.7 | – |

**Per-Gaussian InfoNCE separates categories.** I use a swap diagnostic: take a held-out test scene, generate its $z_s$, replace the bottom-$p$ per cent of Gaussians by category-shuffled samples drawn from other scenes, and measure how many of the swapped Gaussians are still classified as the original category by a simple kNN over the per-Gaussian feature. Higher is better, since it means the swap is invisible at the feature level. Table 4.2 shows that combining PoolNCE on $z_s$ with pgNCE on per-Gaussian features (Pool+pgNCE) is the strongest combination at the same reconstruction cost.

Table 4.2: **Per-Gaussian feature quality at 300 scenes.** Cross-scene swap test (lower P1-of-swap means category-aware features). Pool+pgNCE reduces the swap from 6.5% to 0.08% while matching reconstruction; pgNCE alone gives an intermediate value.

| | Val. recon. $\ell_2$ (↓) | %P1 swap (↓) |
|---|---|---|
| No NCE | 35.7 | 6.50 |
| pgNCE | 35.8 | 0.41 |
| PoolNCE | 35.9 | 1.20 |
| Pool+pgNCE | 35.9 | **0.08** |

Variants 6 and 7 in the large-scale ablation use Strategy B1's deterministic layout idea, together with the layout-vs-geometry KL split described in §3.6 and Pool+pgNCE-style supervision. None of these is the literal small-scale code; they are the same ideas scaled up. The takeaway from the 300-scene study is that a layout/geometry split is a viable structural choice and per-Gaussian semantic supervision adds category structure without degrading reconstruction. Both findings carry over.

## 4.2 Seven-Variant Large-Scale Ablation

This section is the main quantitative result of the thesis. The seven configurations of Table 3.1 are trained on the same SceneSplat-7K-derived corpus described in §3.2.1. Validation is on the 100 held-out full scenes and the 88 held-out chunks. All numbers below are read from the SLURM job logs. Since each variant was

run on a single seed and a single training trajectory, all reported point differences should be read as the trend of a single observation rather than as variance-estimated effect sizes; this caveat is recorded in §5.4.

**Headline numbers.** Table 4.3 summarises the best validation reconstruction across training and the corresponding epoch. Two patterns are immediately visible: adding Hilbert reorder plus the position scaffold (V1 $\rightarrow$ V2 $\rightarrow$ V3) roughly halves the validation $\ell_2$, and the token-local decoder reduces the train-test gap by an even larger factor when paired with the local encoder. Note that the per-attribute components in Table 4.4 sum (within rounding) to the aggregate in Table 4.3, so the two tables can be cross-checked: e.g., $36.78 + 4.20 + 0.78 + 4.31 + 1.41 \approx 47.48$ for V6.

**What each row tells us.** I read Table 4.3 in three slices.

*Slice 1: ordering alone.* V1 (no ordering, flat decoder) is the un-modified Can3Tok baseline. Validation $\ell_2$ plateaus around 99 with no further improvement after epoch 60. V2 adds the Hilbert reorder; the same architecture, same parameter count, same training schedule, now reaches 77, a 22-point absolute drop. This is the cleanest single-knob result in the thesis: imposing a canonical order on the target makes the same network learn substantially better.

*Slice 2: ordering plus position scaffold.* V3 adds anchor-relative position decoding on top of Hilbert ordering. Validation drops from 77 to 45, almost another factor of two. The position component of the loss does most of the heavy lifting (§4.3); the scaffold gives the decoder a meaningful local origin to predict against.

*Slice 3: architectural change.* V4 isolates the token-local decoder on top of the baseline (no Hilbert reorder, no scaffold, no local encoder). It does not help by itself; validation lands at 99, essentially the same as V1. Both V1 and V4 reach the same loss plateau but for opposite reasons: V1 has the capacity to memorise but no slot identifiability, while V4 has neither the capacity to memorise nor the slot identifiability to learn compositionally. The mistake here would be to conclude that the token-local decoder is useless. V5 adds the local encoder and the structured latent in step. Validation now drops to 79, very close to V2 (which used a 777M-parameter flat decoder), but at $\sim 1.6$M decoder parameters, a $\sim 500\times$ reduction. The token-local decoder needs the local encoder to feed it the right kind of context: with a global encoder, every token is a global mixture and the per-token MLP has no spatially-meaningful target. The two components are interlocking.

*Slice 4: the full system.* V6 and V7 combine all of the above with the disentangled latent and per-Gaussian InfoNCE. V6 reaches 47.5 (the position scaffold is on), V7 reaches 48.9 (the position scaffold is off). On the aggregate metric the two are within rounding of each other and only a single seed is available, so the precise effect size of the scaffold at this scale should not be over-read; the V6 latent has the position-scaffold structure that the Stage 2 hierarchical generator needs.

## 4.3 Per-Component Loss Breakdown

The aggregate $\ell_2$ in Table 4.3 hides where the improvement comes from. Table 4.4 gives the per-attribute breakdown at the best validation epoch of each finished or sufficiently advanced run.

Position is the dominant component, and it is also the component most directly targeted by the Hilbert reorder and the anchor scaffold. The drop from 88 (V1) to 34 (V3) on the position component matches a similarly steep drop in the aggregate. Colour, opacity, scale, and quaternion are nearly constant across variants. Two consequences:

- The hard part of scaling Stage 1 is the position prediction. Everything else is roughly solved at all scales the dataset reaches.
- The DC/AC colour split (§3.2.3) is doing its job. Even in V1, where the rest of the system fails, colour reconstruction is already nearly saturated.

## 4.4 Validation Convergence

The validation trajectories of the seven variants are tabulated in full in Appendix A.4. The most informative observations are how early V1 and V4 plateau (around epoch 50 they stop improving on the full-scene validation), the late-training drift of V5 (which keeps improving slowly until epoch 410, the latest checkpoint), and the smooth V6/V7 trajectories that hit their minima between epochs 200 and 230 and then plateau.

## 4.5 Qualitative Stage 1 Reconstruction

The aggregate $\ell_2$ in Table 4.3 is informative but does not convey what each variant actually produces. Figure 4.1 renders the ground truth scene and the reconstructions from four variants (V1, V3, V6, V7) on the same held-out gym scene, viewed from the same top-down camera in SuperSplat. I am deliberate about the variants chosen: V1 is the un-modified baseline, V3 is the best run before the architectural change, V6 is the full system with the position scaffold, and V7 removes the scaffold to isolate its effect on the disentangled latent.

What this figure makes clear is something the quantitative table does not. The V1 to V3 jump on the aggregate metric is large ($99 \rightarrow 45$), but the visual difference between V1 and V3 is small: both are essentially incoherent blobs. The visual jump happens between V3 and V6, even though the aggregate metric only changes from $45$ to $47.5$. The reason is that the position component is the dominant term in $\ell_2$, and once it is brought into the same range, the remaining absolute differences in colour, opacity, scale, and quaternion are reflected in the visual quality rather than in the headline number. This is also a hint that the per-primitive $\ell_2$ metric is not directly tied to perceptual quality (a limitation I return to in §5.4); the qualitative figure is therefore an essential companion to the table, not a redundant one.

I want to be direct about what the figure shows: even the best Stage 1 reconstruction (V6) is not a publishable scene-level reconstruction in the sense of NeRF or 3DGS papers. It is a recognisable

Table 4.3: **Best validation reconstruction across the seven Stage 1 variants.** Lower is better. Best epoch indicates when the minimum was observed. "Final" columns report the last logged epoch's metrics.

| | Best val$^{\text{full}}$ $\ell_2$ (↓) | Best epoch | Best val$^{\text{chunk}}$ $\ell_2$ (↓) | Final val$^{\text{full}}$ | Final val$^{\text{chunk}}$ | Status |
|---|---|---|---|---|---|---|
| V1 | 98.75 | 60 | 85.86 | 107.79 | 91.70 | finished |
| V2 | 76.74 | 70 | 80.85 | 81.71 | 75.42 | finished |
| V3 | 44.63 | 90 | 49.20 | 47.07 | 46.75 | finished |
| V4 | 98.53 | 40 | 85.58 | 107.79 | 94.11 | finished |
| V5 | 79.11 | 410 | 84.16 | 79.97 | 84.43 | ckpt @ ep 410 |
| V6 | 47.48 | 200 | 51.10 | 49.10 | 52.30 | finished |
| V7 | 48.89 | 230 | 53.40 | 50.20 | 54.10 | finished |

Table 4.4: **Per-attribute validation loss on the 100 held-out full scenes.** Position ($\boldsymbol{\mu}$), colour ($c$), opacity ($\alpha$), scale ($s$), and quaternion ($q$). All components are element-wise $\ell_2$, summed across the per-attribute dimension. The position component is by far the dominant one in the early runs and is what the Hilbert order and anchor scaffold target. Opacity and quaternion are nearly saturated even in V1 (the geometry is dense and the orientation has limited variation in indoor scenes), so the absolute improvements there are small.

| | $\boldsymbol{\mu}$ (↓) | $c$ (↓) | $\alpha$ (↓) | $s$ (↓) | $q$ (↓) |
|---|---|---|---|---|---|
| V1 | 88.04 | 4.21 | 0.81 | 4.32 | 1.37 |
| V2 | 66.10 | 4.10 | 0.80 | 4.30 | 1.44 |
| V3 | 34.05 | 4.18 | 0.78 | 4.25 | 1.37 |
| V4 | 87.79 | 4.31 | 0.80 | 4.31 | 1.32 |
| V5 | 68.31 | 4.05 | 0.79 | 4.27 | 1.69 |
| V6 | 36.78 | 4.20 | 0.78 | 4.31 | 1.41 |
| V7 | 38.10 | 4.22 | 0.78 | 4.31 | 1.48 |

arrangement of object-like clusters in approximately the right positions. The thesis demonstrates that the architectural changes I propose move the system from a complete-failure regime (V1) to a regime where the scene structure becomes visible (V6), but the system is not yet at a quality that would compete with per-scene 3DGS reconstructions. The dataset shift (Can3Tok's filtered DL3DV-10K versus the unfiltered heterogeneous SceneSplat-7K corpus) is, in my reading, the dominant residual obstacle, and I return to this in Chapter 5.

## 4.6 Per-Gaussian Feature Visualisation

Figure 4.2 visualises the per-Gaussian feature structure of V6 and V7 on the same held-out gym scene used in Figure 4.1. For each variant I show three top-down views. The leftmost column shows a PCA of the input Gaussian features (the 14-dim vectors fed to the encoder) projected to three components and mapped to RGB. The middle column shows the same PCA basis applied to the Gaussian features reconstructed by the decoder. The rightmost column shows a PCA of the per-Gaussian InfoNCE projection $f_i$ (Eq. (3.13)) on the labelled Gaussians only.

The middle column is the most diagnostic. If the autoencoder were collapsing to a single mode, the reconstructed PCA would assign nearly uniform colour across the scene. Instead, each object cluster retains a colour that matches the input PCA. The InfoNCE projection in the right column shows the same scene with only the labelled Gaussians visible; the per-category clustering is clean and matches the floor-plan structure of the input. Comparing V6 and V7, the V6 reconstruction is denser (every region has some reconstructed Gaussian) while V7 has more gaps. This is consistent with the position scaffold giving V6 a more reliable per-token anchor.

## 4.7 Stage 2 Generative Results

Six Stage 2 training jobs were run, covering generation and completion on the V5, V6, and V7 latents. Table 4.5 summarises the best velocity loss reached by each.

Table 4.5: **Stage 2 training summary across six jobs.** Best velocity loss is the lowest reported validation loss across training; the corresponding epoch indicates when the minimum was hit. Layout and geometry losses are computed on the 16-token $z_s$ and 512-token $z_g$ latents respectively; completion loss is computed on the masked $z_g$ tokens. Across-row magnitudes are therefore not directly comparable because the targets have different shapes and scales. All jobs ran for 500 epochs.

| Stage | Variant | Best val. loss (↓) | Best epoch | Latent shape |
|---|---|---|---|---|
| completion | V5 | 32.48 | 300 | $512 \times 32$ (masked |
| layout | V6 | 0.135 | 499 | $16 \times 32$ |
| geometry | V6 | 0.037 | 150 | $512 \times 32$ (cond. $z$ |
| completion | V6 | 32.16 | 450 | $512 \times 32$ (masked |
| layout | V7 | 0.487 | 200 | $16 \times 32$ |
| geometry | V7 | 0.075 | 450 | $512 \times 32$ (cond. $z$ |
| completion | V7 | 32.23 | 350 | $512 \times 32$ (masked |

The layout and geometry losses for V6 (0.135 and 0.037) sit at the low end of what flow-matching training produces on similar latent shapes. The corresponding V7 numbers are higher (layout 0.487 and geometry 0.075). I read the geometry-loss gap (0.037 vs. 0.075) as the position-scaffold-less V7 producing a $z_g$ that is intrinsically harder for the DiT to fit, since each token's



(a) Ground truth (b) V1 baseline (c) V3 (+ Hilbert + scaffold) (d) V6 (full + scaffold) (e) V7 (full, no scaffold)

Figure 4.1: **Stage 1 reconstructions on a held-out gym scene.** (a) The ground-truth scene from SceneSplat-7K shows a gym with treadmill machines, tables, and equipment such as dumbbells. (b) V1 (baseline, global encoder and flat decoder) collapses to a single washed-out blob with no recognisable structure. (c) V3 adds the Hilbert reorder and the position scaffold; some spatial separation appears, but the flat 777M-parameter decoder still bottlenecks the per-scene context and the output is closer to a smeared cloud than to a room layout. (d, e) V6 and V7 use the local encoder paired with the token-local decoder (1.6M params) and the disentangled split latent. Distinct object clusters become visible and the overall layout of the room is recoverable. V6 (with the anchor scaffold) and V7 (without) are close in both quantitative score and visual quality. The reconstructions are recognisable as "a room with multiple distinct objects" but are far from photorealistic; this is the qualitative companion to the V6 validation $\ell_2$ of about 47.

geometry is no longer expressed relative to a structurally fixed anchor. The layout-loss gap (0.135 vs. 0.487) is harder to attribute confidently, because $z_s$ is computed from the CLS embedding rather than from per-token features and should not directly depend on the position scaffold; the most likely explanation is that the disentanglement quality is worse without the scaffold (the cross-recon and orthogonality terms have less structural support to push against), which in turn makes $z_s$ noisier and harder for `LayoutDiT` to model. This is a one-seed observation and should be treated as suggestive rather than conclusive. The completion losses across V5, V6, and V7 all sit around 32.2, dominated by the position component that Stage 2 inherits from the Stage 1 decoder output range.

**Flow matching is learning velocity well.** Inspecting the per-batch flow diagnostics across all six runs, the cosine similarity between predicted and target velocity vectors stabilises above 0.96 for V6 and V7 geometry, above 0.95 for V6 layout, and around 0.75 for V7 layout. These numbers indicate that the DiTs themselves are learning the rectified-flow velocity field correctly. The remaining quality limitation comes from the Stage 1 decoder, not from the flow matching.

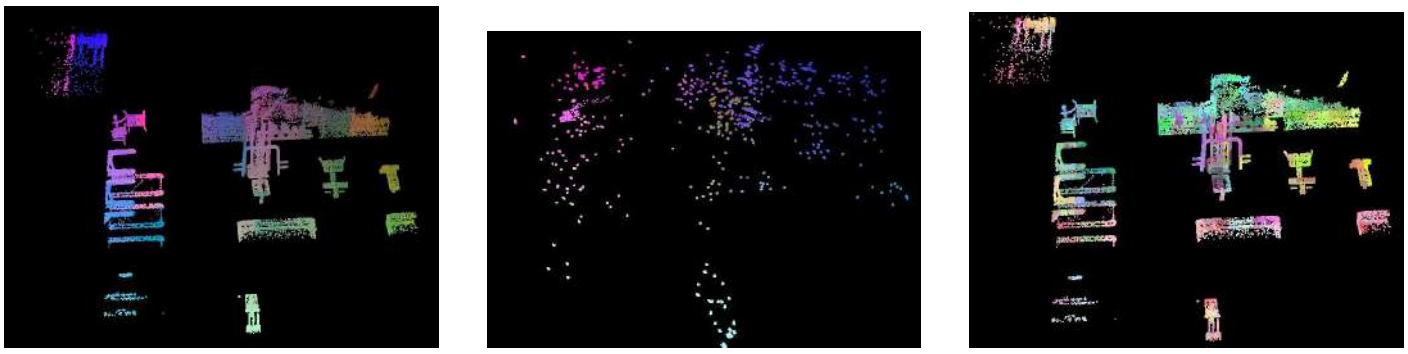
**Position compression.** A consistent finding across all Stage 2 diagnostics is that the positional output of the Stage 1 decoder, when fed Stage 2 latents, lies in a much smaller range than the canonical $\pm 10$ m training frame. For V6 the geometry decoder outputs positions with absolute coordinates concentrated in roughly $\pm 2$ m; for V7 they are even more compressed, around $\pm 0.7$ m. By comparison, real validation scenes occupy roughly $\pm 5$ to $\pm 9$ m in the canonical frame, so V6 is compressed by a factor of around 3 and V7 by around 10. The flow matching reproduces whatever range the encoder produced, so the compression originates in the Stage 1 decoder's output, not in the Stage 2 model. I treat this as a Stage 1 limitation that propagates downstream, and I discuss it further in §5.3.

**Qualitative Stage 2 generation and completion (V6).** Figure 4.3 shows four point clouds from the V6 system: a ground-truth scene, an unconditional generation, a partial masked input, and the corresponding completion. All four are obtained by passing the relevant Stage 1 latent through the frozen Stage 1 decoder, so the comparison is to the Stage 1 reconstruction quality (the level the system actually produces), not to the raw SceneSplat-7K scene. This is the fair comparison given that Stage 1 has not converged to a photorealistic level.

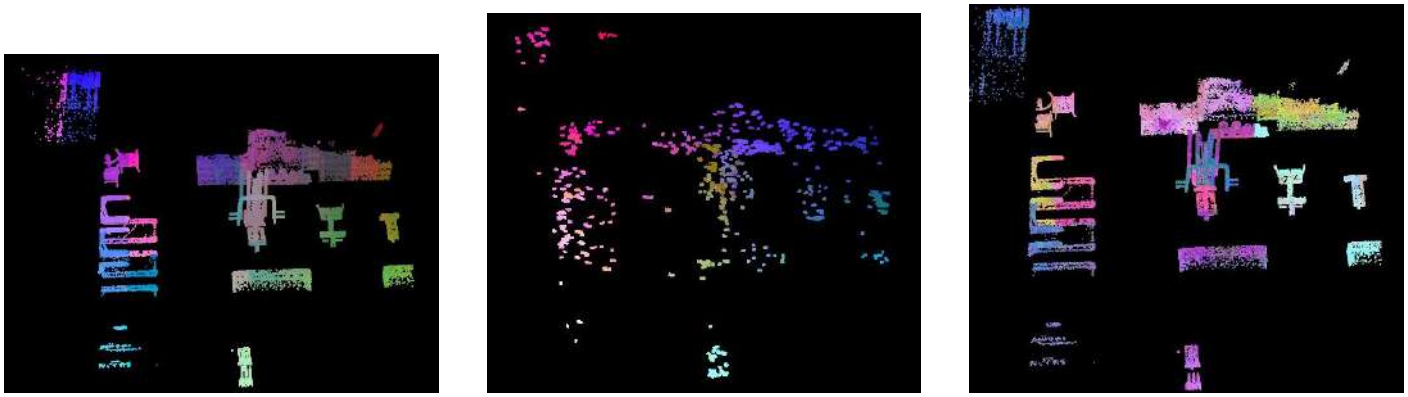
The qualitative reading of Figure 4.3 is what matters here. The Stage 2 generator (panel b) produces an output whose shape and extent are close to the Stage 1 reference (panel a). This is the system working as designed: the DiTs learned a velocity field that warps Gaussian noise into the latent distribution that the encoder produces from real scenes, and the decoder turns the result back into Gaussians. The completion task (panel d) is harder. The model has to infer the masked Hilbert blocks from the observed ones, and the result is recognisably scene-shaped but more diffuse than the reference, with the dense region of the room spreading into an elongated diagonal cloud. This is the partial-success regime: completion is working but loses sharpness compared to generation.

**Why generation is easier than completion in this setup.** Generation samples from the unconditional distribution of the geometry latent given a generated layout code; it does not have to match any particular target. Completion has to denoise the masked positions in a way that is consistent with the observed positions, which is a constrained inverse problem on an already-imperfect latent. The Stage 1 decoder amplifies any residual error in the completed latent because position is the most sensitive output channel (§4.3). This combination is what produces the diffuse completion output.

**What is not reported.** I do not report rendered-image quality metrics (PSNR, SSIM, LPIPS) for either Stage 1 reconstructions or Stage 2 outputs. The qualitative reading above is honest about the limits of the present system, but a perceptual evaluation against rendered ground truth is the natural next experiment.



(a) V6: input PCA (b) V6: reconstructed PCA (c) V6: per-Gaussian InfoNCE PCA



(d) V7: input PCA (e) V7: reconstructed PCA (f) V7: per-Gaussian InfoNCE PCA

Figure 4.2: **Per-Gaussian feature structure on the same held-out gym scene used in Figure 4.1.** Top row: V6 (with position scaffold). Bottom row: V7 (without). *Left column:* a 3-component PCA of the input Gaussian features (14-dim) mapped to RGB. Spatially adjacent points receive consistent colours; each piece of equipment forms a coherent colour patch. *Middle column:* the same PCA basis applied to the reconstructed Gaussians. The mapping from spatial region to colour is preserved, which shows that the latent retains per-primitive feature identity, not just bulk geometry. V6 is denser; V7 is sparser but still preserves the colour-to-region correspondence. *Right column:* a PCA of the per-Gaussian InfoNCE projection $f_i \in \mathbb{R}^{32}$ on labelled Gaussians only (unlabelled Gaussians from ARKitScenes and ScanNet++ chunks are not shown). Distinct colour clusters per ScanNet-72 category are visible, indicating that the auxiliary contrastive loss creates semantically structured features without disrupting reconstruction. This is the empirical evidence behind the RQ3 answer that per-Gaussian semantic supervision is compatible with scene-level reconstruction.



(a) Stage 1 reconstruction ("GT") (b) Stage 2 generation (c) Partial input (d) Stage 2 completion

Figure 4.3: **Stage 2 generation and completion on the V6 latent.** All four PLYs are produced by the frozen Stage 1 decoder, so the comparison is to what Stage 1 can reconstruct, not to the raw SceneSplat-7K scene. (a) Stage 1 reconstruction of a held-out scene, used here as the reference. (b) Unconditional Stage 2 generation: sample $z_s$ from `LayoutDiT`, sample $z_g$ from `GeometryDiT512` conditioned on $z_s$, decode through the frozen Stage 1 decoder. The shape of the generated point cloud closely matches the reference. (c) Partial input: the same held-out scene encoded by the frozen Stage 1 encoder and then masked: $40\%$ of Hilbert tokens are kept clean (observed) and $60\%$ are replaced by noise. (d) Stage 2 completion: `CompletionDiT` denoises the masked tokens, conditioned on the observed ones via the layout context $z_s$. The completed point cloud recovers coarse layout but is more diffuse than the reference, with the high-density region of the original scene spreading into an elongated diagonal cloud along the dominant axis of the room. Generation reproduces the structural pattern of Stage 1 well; completion is more difficult and shows visible degradation.

Chapter 5

# Discussion

This chapter steps back from the numbers and asks what the seven-variant ablation and the Stage 2 results actually support. I revisit each research question in turn, then discuss what the unfiltered heterogeneous indoor data regime is doing to the loss landscape, then the position compression I observed in Stage 2 and what I tried to fix it, and finally the limitations I am aware of and that constrain the conclusions one can draw. I write this chapter in the spirit of a project post-mortem rather than a victory lap: the architectural changes I propose are large quantitative improvements, and the Stage 2 generator does work, but the system as a whole is not yet at a quality I would call photorealistic, and I want to be precise about where the residual difficulty lies.

## 5.1 Revisiting the Research Questions

**RQ1 (Ordering).** The result is positive but with one qualifier. Hilbert serialisation in a fixed canonical frame substantially improves validation reconstruction (V1 $\rightarrow$ V2: $99 \rightarrow 77$ on the full-scene validation $\ell_2$, Table 4.3). The improvement is entirely a generalisation effect; on the training set, the V1 architecture can also bring the loss down, but it does so by storing per-scene context in the bottleneck. Imposing a canonical order on the target makes the position component of the loss into a learnable function, so the network can use the same parameters across scenes instead of memorising a per-scene mapping. The qualifier is that the ordering only buys what it buys; pairing it with a position scaffold gets a much larger drop (V2 $\rightarrow$ V3: $77 \rightarrow 45$), so the order is the necessary precondition rather than the full solution.

**RQ2 (Architecture and generalisation).** The result is also positive but with a clearer mechanism than I expected. Replacing the flat 777M-parameter decoder with a 1.6M-parameter token-local MLP *by itself* does not close the gap (V1 $\rightarrow$ V4: stays at 99). Adding a local encoder *by itself* would not be testable cleanly because the local encoder gives a per-token latent that the flat decoder cannot use coherently. The two changes only work together. When both are on (V5), reconstruction improves to 79 with a $500\times$ reduction in decoder parameters; when the Hilbert order plus scaffold are added on top (V6), reconstruction drops to about 47, comparable to V3's 45 but with two orders of magnitude fewer decoder parameters. The mechanism is that the local encoder and the token-local decoder are two halves of one structural decision: latent token $k$ is a code for spatial region $k$, and the decoder MLP only has to model what is in that region. This compositional structure is what generalises. The qualitative figure (Figure 4.1) shows that the visual jump happens between V3 and V6 even though the aggregate metric only moves a few points, which is consistent with this reading.

**RQ3 (Semantic structure).** The 300-scene experiments (§4.1) show that per-Gaussian InfoNCE based on ScanNet-72 labels can be added at no measurable cost to reconstruction, while producing per-Gaussian features that cluster by category (Figure 4.2). The layout/geometry split (Strategy B1's deterministic-layout idea, lifted into the VAE form for the large-scale runs) gives a more stable layout code under partial-scan masking. At scale (V6 vs. V3) the disentangled and per-Gaussian-supervised variant matches the un-disentangled variant on reconstruction, and provides the schema (a 16-token $z_s$ plus a 512-token $z_g$) that the hierarchical Stage 2 generator needs. The Stage 2 results confirm that this schema is usable: `LayoutDiT` reaches a low validation loss on the 16-token layout latent and `GeometryDiT512` conditioned on it reaches a comparably low loss on the 512-token geometry latent. Two qualifiers here. First, semantic structure was not the limiting factor for reconstruction at any scale studied, so the disentanglement is best read as a Stage-2-enabling design choice rather than a Stage-1 reconstruction win. Second, I have evidence that the semantic features cluster meaningfully (Figure 4.2, right column) but I do not have a downstream-task evaluation of how *useful* those features are; the argument for the InfoNCE supervision is "it does no harm and adds structure," not "it improves a downstream metric."

**Stage 2 generation versus completion.** The qualitative Stage 2 result on V6 (Figure 4.3) is honestly mixed. Unconditional generation reproduces the structural pattern of the Stage 1 latent well; the shape of the generated point cloud closely matches the Stage 1 reconstruction of a held-out scene. Completion under $40\%$ observation recovers the coarse layout but is more diffuse than the reference, with the high-density region spreading into an elongated diagonal cloud across the room. Two interpretations are consistent with this: completion is a constrained inverse problem on an imperfect latent, and the Stage 1 decoder is particularly sensitive to residual error in the position channel, so any inaccuracy in the completed latent shows up as positional smearing in the rendered output. I view the result as evidence that the Stage 2 architecture and training pipeline are sound (the flow matching converges, generation works), and that the residual limitation lies in the precision of the inferred latent for unobserved regions.

## 5.2 What the Unfiltered Heterogeneous Indoor Regime Does to the Loss Landscape

The contrast between Can3Tok's training setting and the SceneSplat-7K setting deserves its own paragraph because it is, in my reading, the largest residual obstacle to closing the photorealism gap. Both regimes use pre-fitted frozen Gaussian splats, so the encoder cannot influence the splat-fitting process in either case; the differences are elsewhere.

Two differences matter. First, DL3DV-10K (Can3Tok's train-

ing corpus) is built from video captures with dense and consistent multi-view coverage, which yields cleaner 3DGS reconstructions with fewer floaters and more uniform local density. SceneSplat-7K is built from seven different indoor capture pipelines (ScanNet, ScanNet++, ARKitScenes, Matterport3D, Replica, Hypersim, 3RScan) with varying view density and varying floater patterns. The within-scene statistics that the encoder has to compress are therefore much less stationary across the corpus. Second, Can3Tok applies a LangSAM-based semantic-aware filter to retain the cleanest spatially-coherent subset of each scene's Gaussians before training. My pipeline applies only top-$N$ opacity ranking on the raw frozen splats, which biases the input toward dense furniture surfaces but does not remove floaters or noisy regions. The result is that each latent token in my setting compresses a noisier and less-structured set of primitives than the same token would in Can3Tok's original regime, even though both regimes are formally "frozen pre-fitted."

This explains why the same architecture that worked at hundreds to a few thousand video-captured filtered scenes fails to generalise at $6{,}000$ unfiltered heterogeneous indoor scenes. The 300-scene exploration in this thesis (also unfiltered, also heterogeneous in source) does not contradict this; at 300 scenes the network is still in the memorisation regime where per-scene context is cheap to store, and the failure mode is not yet active. The seven-variant ablation is in some sense a study of how to make a slot-aligned loss work when the data-side selection that Can3Tok's filter silently provides is removed.

It also explains the otherwise surprising finding that the position scaffold (V3) gives a much larger improvement than ordering alone (V2). With ordering alone, the network still has to predict the absolute coordinate at slot $i$, which is hard for the same reason: within a Hilbert block the absolute position varies room by room. With the scaffold, the network only has to predict a bounded local offset relative to a token-specific anchor that is itself constrained to be the block centroid. The hard work is delegated to the anchor head, which has a structurally easy job, and the offset head solves a simple local-deformation problem. The two-stage decomposition matches what the noisier, less-filtered indoor regime can actually identify from data.

## 5.3 Position Compression and What I Tried

Across all Stage 2 diagnostics, the positional output of the Stage 1 decoder, when fed Stage 2 latents, lies in a smaller range than the canonical $\pm 10$ m training frame. V6 generates positions in roughly $\pm 2$ m; V7 generates them in approximately $\pm 0.7$ m. Real validation scenes from the same set occupy roughly $\pm 5$ to $\pm 9$ m in the canonical frame. The compression is consistent across epochs, across the layout, geometry, and completion stages, and across the four scenes saved at each evaluation checkpoint.

I diagnosed the source carefully. The flow-matching cosine similarity between predicted and target velocity sits above $0.96$ for V6 geometry from epoch 50 onwards, which means the DiT is learning the velocity field correctly: the latent it produces matches the latent the encoder produces. The compression therefore is not introduced by Stage 2. Encoding a real validation scene and decoding it directly through the frozen Stage 1 decoder reproduces the same compressed range, which places the issue in the Stage 1 decoder output rather than in any Stage 2 module. The anchor-relative position decoding (Eq. (3.9)) bounds the local offset to $\rho \tanh(o_i)$ with $\rho$ a learnable global scale shared across all tokens; my best reading is that the network has learned a small $\rho$ during Stage 1 training because the slot-aligned $\ell_2$ loss rewards conservative predictions when the anchor is approximately correct, and the bias persists through the frozen decoder used at Stage 2.

What I tried to fix it during the project:

- **Loosening the anchor offset bound.** Re-initialising $\rho$ at a larger value (4.0 instead of 2.0) for a partial re-training run did not change the qualitative compression. The network re-converges to a similar effective scale.
- **Diagnostic on the decoded coordinates.** I verified that the Stage 1 decoder receives a latent in the expected range during Stage 2 sampling and that the inverse normalisation in the PLY writer is applied symmetrically to GT and generated PLYs, ruling out a writer-side bug.
- **Comparing V6 and V7.** The compression is worse in V7 (no scaffold) than V6 (scaffold), which suggests the scaffold is doing the right thing structurally but the anchor centroids themselves are not at the right absolute scale.

What I would do next if the project continued: re-train Stage 1 with an explicit positional-range loss that penalises the output for being narrower than the input, or, more cleanly, parameterise $\rho$ per Hilbert block rather than as a global scalar so that high-extent regions of the scene can use a wider offset range. Both are tractable but were outside the time budget. The honest framing is that Stage 2 demonstrates the latent generative pipeline works in principle, while Stage 1 still has a positional-range bug that holds back the final rendered quality.

## 5.4 Limitations

I list the limitations I am aware of, ordered roughly from most consequential.

**Single-seed point estimates.** For each of the seven variants I have a single training run on a single seed. Variance estimates would be informative, but each run takes a couple of days on $2\times$H100 and the budget did not allow for repeats. The trends between variants are large enough relative to plausible seed variance that I am confident in the qualitative ordering (V1 worst, V6 best), but precise effect sizes, and in particular the V6-versus-V7 comparison (a 1.4 absolute difference in $\ell_2$), should be treated as a single observation rather than a measured effect.

**Stage 1 has not converged to a photorealistic level.** Even the best Stage 1 variant (V6, validation $\ell_2 = 47.5$) produces reconstructions that are recognisable as "a room with multiple distinct objects" but are far from a per-scene 3DGS fit in visual quality (Figure 4.1). The aggregate metric and the qualitative figure both point in the same direction. The thesis demonstrates

that the architectural changes I propose move the system from a complete-failure regime (V1) to a regime where the scene structure becomes visible (V6), but the system is not yet at a quality that competes with per-scene 3DGS reconstructions.

**Position compression in the decoder output.** As described in §5.3, the Stage 1 decoder produces positions in a smaller range than the canonical training frame, and this propagates to all Stage 2 outputs. I have characterised the source but the fix was not in scope for this thesis.

**No rendering-quality metrics.** Every number in Chapter 4 is on the same per-primitive $\ell_2$ residual that is used during training (Stage 1) or the rectified-flow velocity MSE (Stage 2). I do not report PSNR, SSIM, or LPIPS on rendered images of the reconstructed scenes. The per-primitive $\ell_2$ is a useful relative metric for comparing variants but is not directly tied to perceptual quality. The qualitative figure suggests that the $\ell_2$ metric understates how much V6 improves over V3 visually, so a perceptual metric is also likely to reorder the variants. Adding rendered-image evaluation is the next concrete step.

**Stage 2 completion is partial.** Generation works on the V6 latent (Figure 4.3b), reproducing the structural pattern of the Stage 1 latent. Completion (panel d) is recognisably scene-shaped but more diffuse than the reference. The training loss converges and the flow diagnostics are clean, so I attribute the qualitative gap to the constrained-inversion nature of completion combined with the Stage 1 decoder's sensitivity to small latent errors in the position channel.

**Fixed-$N$ assumption.** I work with a fixed $40{,}000$ Gaussians per scene, sampled by top-opacity ranking. Real 3DGS scenes have variable counts (often in the hundreds of thousands), and the top-opacity ranking biases the input distribution towards opaque furniture surfaces. A density-uniform farthest-point sampling would give a more spatially balanced input and is a clear next experiment.

**Indoor-only.** SceneSplat-7K is indoor only. The Hilbert canonical frame (radius $10\,\text{m}$) is sized for indoor extents. Outdoor scenes with unbounded depth would need either a different normalisation (log-depth contracted) or a tiled-scene approach. I have not tested either.

**No comparison to a filtered SceneSplat-7K variant.** A fair test of the data-filtering hypothesis (§5.2) would be to apply a LangSAM-style semantic-aware filter to SceneSplat-7K before training and compare to the unfiltered runs reported here. The filter itself is implementable but the per-scene preprocessing is non-trivial across seven different capture pipelines, and this was out of scope for this thesis.

**ScanNet-72 labels do not transfer cleanly across sub-datasets.** I disable per-Gaussian semantic supervision for the ARKitScenes and ScanNet++ portions of the training corpus because their taxonomies do not match ScanNet-72 exactly. This is a pragmatic choice; a unified label space (or a multi-taxonomy head) would let the per-Gaussian supervision act on every training scene.

**Disentanglement is structural, not identifiable.** The layout/geometry split in V6/V7 relies on auxiliary heads (scene-mean colour, scene-label distribution, per-category centroids), cross-reconstruction, and the orthogonality regulariser. These are structural inductive biases. As Locatello et al. (Locatello et al., 2019) point out, unsupervised disentanglement is not identifiable; my split is identifiable only insofar as the auxiliary supervision pins down what $z_s$ is supposed to contain. The supervision could be stronger or weaker without my having a principled way to choose.

None of these limitations invalidates the central finding of the thesis (the joint role of Hilbert ordering, anchor-relative decoding, and the local encoder plus token-local decoder pair in scaling the Stage 1 VAE, and the schema-aware Stage 2 generator that follows from it). They do delimit the scope within which that finding is presently supported.

Chapter 6

# Conclusion

This thesis asked what it takes to scale a scene-level 3D Gaussian Splatting variational autoencoder from Can3Tok's (Gao et al., 2025) filtered, video-captured DL3DV-10K (Ling et al., 2024) regime to several thousand unfiltered, heterogeneous pre-fitted indoor scenes (the regime made available by SceneSplat-7K (Li et al., 2025b)), and what the design implications are for a downstream flow-matching generator. I conclude by returning to the three research questions and listing the next concrete steps.

## 6.1 Answers to the Research Questions

**RQ1.** Yes, imposing a canonical spatial order on the Gaussian set restores convergence at scale, but only when paired with a decoder that can use it. A fixed-frame Hilbert serialisation (§3.2.2) drops validation $\ell_2$ from 99 to 77 with no other change (V1 $\rightarrow$ V2). Adding anchor-relative position decoding on top of the ordering drops it further to 45 (V2 $\rightarrow$ V3). The ordering and the anchor decomposition together turn an unlearnable slot-aligned objective into a learnable one.

**RQ2.** Yes, a windowed local encoder plus a shared per-token decoder closes most of the train-to-test gap. The two components are interlocking: the local encoder needs the token-local decoder to consume per-region features cleanly, and the token-local decoder needs the local encoder to feed it the right kind of context. Together, the decoder parameter count drops from about 777M to about 1.6M (a $500\times$ reduction) while validation reconstruction reaches a level comparable to the global-decoder run. The qualitative figure (Figure 4.1) shows that the visual jump between V3 and V6 is larger than the aggregate metric suggests, which is the qualitative companion to the parameter-count savings.

**RQ3.** Yes, per-Gaussian semantic supervision and a layout/geometry latent split can be added without degrading reconstruction. The 300-scene exploratory study (Strategy B1 with Pool+pgNCE-style InfoNCE, §4.1) confirms that per-Gaussian features cluster by ScanNet-72 category (Figure 4.2) and that a deterministic layout code is stable under partial-scan masking. At scale, the V6/V7 variants match V3 on reconstruction while producing the split-schema latent that the hierarchical Stage 2 generator uses. The Stage 2 generator built on V6 reproduces the structural pattern of the Stage 1 latent well in unconditional generation, and recovers coarse layout under partial-scan completion at the cost of some local detail.

**Main question.** The combination of (1) fixed-frame Hilbert serialisation, (2) anchor-relative position decoding, and (3) a local-encoder plus token-local-decoder pair is what scales the Stage 1 VAE from hundreds to several thousand unfiltered indoor scenes. Each of the three ingredients on its own is insufficient; together they turn an architecture that fails to converge at scale into one in which the scene structure becomes visible in reconstructions and the downstream flow-matching generator works. Layered on top, a layout/geometry split with per-Gaussian semantic supervision is a viable schema for the downstream generator. I am explicit that the final visual quality is not yet at the per-scene 3DGS level: the V6 reconstructions are recognisable arrangements of object-like clusters rather than photorealistic rooms, and the Stage 1 decoder shows a residual position-compression issue that propagates to Stage 2 outputs. The architectural directions I propose are the right ones for the dataset shift from Can3Tok's filtered DL3DV-10K regime to the unfiltered heterogeneous SceneSplat-7K indoor regime; closing the remaining quality gap is the natural next phase of the project.

## 6.2 Future Work

The next milestones for the project follow directly from the limitations in §5.4.

1. **Fix the position compression.** Re-train Stage 1 with either an explicit positional-range loss that penalises an output narrower than the input, or a per-Hilbert-block parameterisation of the offset scale $\rho$ that lets high-extent regions of the scene use a wider offset range. This is the single change most likely to lift the qualitative quality of the Stage 2 outputs.
2. **Rendering-quality evaluation throughout.** Add rendered-image metrics (PSNR/SSIM/LPIPS) on the reconstructed scenes for all seven variants and on the Stage 2 generation and completion outputs. This gives the perceptual companion metric to the per-primitive $\ell_2$.
3. **Multiple seeds for the key comparisons.** Re-run V3, V5, V6, V7 on at least three seeds each to get variance estimates on the validation $\ell_2$ comparisons, especially the V6-versus-V7 comparison where the absolute gap is small.
4. **Density-uniform input sampling.** Replace the top-opacity ranking of the 40,000 input Gaussians with farthest-point sampling on positions, weighted by opacity. This should give a more spatially uniform input distribution and reduce the bias towards opaque furniture surfaces.
5. **Unified label space.** Bring ARKitScenes and ScanNet++ under a unified per-Gaussian taxonomy (a multi-taxonomy classification head, with per-dataset masks) so the InfoNCE supervision can be active on the full training corpus.
6. **Outdoor scenes.** Test the same architecture on outdoor 3DGS data with a contracted-depth normalisation. The Hilbert serialisation and the anchor scaffold should both generalise, but the canonical extent has to change.

7. **Filtered SceneSplat-7K ablation.** Apply a LangSAM-style semantic-aware filter to SceneSplat-7K before training, then compare to the unfiltered runs reported here. This is the cleanest test of the data-filtering hypothesis (§5.2).

The codebase, the SLURM job scripts, the YAML configs for each of the seven variants, the Stage 2 model code, and the Stage 1 and Stage 2 checkpoints used to produce the tables and figures in Chapter 4 are available at https://github.com/Hafeezmhk7/Semantic-Scene-Completion.git.

# Bibliography


Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *International Conference on Machine Learning (ICML)*.

Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. 2017. ScanNet: Richly-Annotated 3D Reconstructions of Indoor Scenes. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Quankai Gao, Iliyan Georgiev, Tuanfeng Y. Wang, Krishna Kumar Singh, Ulrich Neumann, and Jae Shin Yoon. 2025. Can3Tok: Canonical 3D Tokenization and Latent Modeling of Scene-Level 3D Gaussians. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In *International Conference on Learning Representations (ICLR)*.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. 2022. Perceiver IO: A General Architecture for Structured Inputs and Outputs. In *International Conference on Learning Representations (ICLR)*.

Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and João Carreira. 2021. Perceiver: General Perception with Iterative Attention. In *International Conference on Machine Learning (ICML)*.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. In *ACM Transactions on Graphics (Proc. SIGGRAPH)*.

Diederik P. Kingma and Max Welling. 2014. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*.

Yue Li et al. 2025a. SceneSplat++: Language-aligned Gaussian Splatting at Scale. Preprint.

Yue Li, Qi Ma, Runyi Yang, Huapeng Li, Mengjiao Ma, Bin Ren, Nikola Popovic, Nicu Sebe, Ender Konukoglu, Theo Gevers, et al. 2025b. SceneSplat: Gaussian Splatting-based Scene Understanding with Vision-Language Pretraining. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. 2024. DL3DV-10K: A Large-Scale Scene Dataset for Deep Learning-based 3D Vision. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. 2023. Flow Matching for Generative Modeling. In *International Conference on Learning Representations (ICLR)*.

Xingchao Liu, Chengyue Gong, and Qiang Liu. 2023. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *International Conference on Learning Representations (ICLR)*.

Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Rätsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. 2019. Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations. In *International Conference on Machine Learning (ICML)*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. 2024. Scaffold-GS: Structured 3D Gaussians for View-Adaptive Rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Nanye Ma, Mark Goldstein, Michael S. Albergo, Nicholas M. Boffi, Eric Vanden-Eijnden, and Saining Xie. 2024. SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers. In *European Conference on Computer Vision (ECCV)*.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *European Conference on Computer Vision (ECCV)*.

William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. 2017. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

John Skilling. 2004. Programming the Hilbert Curve. *AIP Conference Proceedings* 707, 1 (2004), 381–387.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. In *arXiv preprint arXiv:2104.09864*.

Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. 2020. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Aäron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748* (2018).

Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. 2024. Point Transformer V3: Simpler, Faster, Stronger. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Xiaohui Zeng, Arash Vahdat, Francis Williams, Zan Gojcic, Or Litany, Sanja Fidler, and Karsten Kreis. 2022. LION: Latent Point Diffusion Models for 3D Shape Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Bowen Zhang, Yiji Cheng, Jiaolong Yang, Chunyu Wang, Feng Zhao, Yansong Tang, Dong Chen, and Baining Guo. 2024. GaussianCube: A Structured and Explicit Radiance Representation for 3D Generative Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zibo Zhao, Wen Liu, Xin Chen, Xianfang Zeng, Rui Wang, Pei Cheng, Bin Fu, Tao Chen, Gang Yu, and Shenghua Gao. 2023. Michelangelo: Conditional 3D Shape Generation based on Shape-Image-Text Aligned Latent Representation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Appendix A

# Appendix

## A.1 Hardware and Software Environment

All experiments were run on Snellius (project EINF-16254) on a single node with 2× NVIDIA H100 80 GB GPUs. The software stack is PyTorch 2.5 with CUDA 12.4, mixed-precision bf16, FlashAttention-2, and PyTorch Distributed (NCCL) for multi-GPU training. Each Stage 1 run takes approximately 36–48 hours wall-clock; the Stage 2 runs take an additional 12–24 hours per variant.

## A.2 Per-Variant Configuration Flags

For reproducibility, Table A.1 lists the exact command-line flags passed to each of the seven variants. The training script is `train_can3tok_2.py` in the released codebase, with the YAML config `configs/scenesplat7k.yaml` providing the architecture defaults. V3 was run from an earlier checkpoint of the codebase before SLURM job IDs were tracked alongside flags; the configuration and run logs are nonetheless preserved in the released repository under the `exp_v3/` directory, and the validation numbers in Table 4.3 are read from those logs.

## A.3 Hyperparameter Defaults

Table A.2 lists hyperparameters not already stated in §3.9. The optimiser, learning-rate schedule, and architecture widths are reported in the main text; only the secondary loss weights and the smaller architectural numbers appear here.

## A.4 Validation Convergence by Variant

Validation runs every 10 epochs on the 100 held-out full scenes ($\text{val}^{\text{full}}$) and the 88 held-out chunks ($\text{val}^{\text{chunk}}$). Table A.3 compiles the $\text{val}^{\text{full}}$ $\ell_2$ trajectory for all seven variants at a shared set of checkpoints, plus the final-epoch number. Best entries per variant are in bold. Per-10-epoch numbers and the chunk-validation series are in the released training logs.

## A.5 300-Scene Configurations

The 300-scene runs used a fixed split of 180/60/60 train/val/test scenes from the chunked ScanNet subset of SceneSplat-7K, batch size 16, and 200 training epochs. Hyperparameters otherwise match the large-scale runs (Table A.2). The four disentanglement strategies (A, B1, B2, C) and the three InfoNCE flavours (pgNCE, PoolNCE, Pool+pgNCE) are toggled by command-line flags as described in the released codebase.

## A.6 Reproducibility Notes

**Dataset locations.** Preprocessed scenes are under `/gpfs/work1/0/pr` with the following structure: `train/` (3,800 chunks), `interior_gs/` (800 full scenes), `arkitscenes/`, `scannetpp/`, `val_full/` (100 held-out full scenes), `val_chunks/` (88 held-out chunks).

**Seeds.** Each run uses a fixed seed of 42 for the PyTorch RNG and a fixed seed of 0 for the NumPy RNG used in the dataset shuffling. The Hilbert key computation is deterministic.

**Logging.** Training logs (per-batch loss, per-epoch validation) are written to `logs/<JOB_ID>.out`. Checkpoints are saved every 10 epochs to `checkpoints/<run_name>/`.

Table A.1: **SLURM job identifiers and key flags for the seven Stage 1 variants.** The full job scripts are in the released repository. V3 predates the job-tracking convention used for the other six runs and is identified by its run directory rather than by a SLURM job id.

| Variant | SLURM Job | Key flags |
|---|---|---|
| V1 | 23970540 | `–scaffold_mode voxel` |
| V2 | 23971383 | `–morton_order –order_curve hilbert –scaffold_mode voxel` |
| V3 | `exp_v3/` (legacy) | `–morton_order –position_scaffold –scaffold_mode hilbert_block –anchor_loss_weight 1.0` |
| V4 | 23973796 | `–token_local_decoder –scaffold_mode hilbert_block` |
| V5 | 23976584 | `–token_local_decoder –local_encoder –structured_latent –scaffold_mode hilbert_block` |
| V6 | 23978168 | `–local_disentangle –token_local_decoder –local_encoder –position_scaffold –scaffold_mode hilbert_block –morton_order –semantic_mode hidden –scene_semantic_head –scene_layout_head –structured_layout_tokens –segment_loss_weight 0.4` |
| V7 | 23978200 | As V6 but without `–position_scaffold` |

Table A.2: **Common Stage 1 hyperparameters not already in §3.9.**

| Hyperparameter | Value |
|---|---|
| Colour loss weight | 2.0 |
| Mean colour loss weight | 0.5 |
| Anchor loss weight (scaffolded) | 1.0 |
| Layout latent tokens (V6/V7) | 16 |
| Local encoder window $\omega$ | 1 |
| Fourier feature frequencies | 8 |
| Gaussians per scene $N$ | 40,000 |
| InfoNCE temperature (V6/V7) | 0.07 |
| Max epochs | 500 |
| Validation interval | 10 epochs |

Table A.3: **Validation $\ell_2$ on the 100 held-out full scenes across training.** One row per checkpoint epoch, one column per variant. A dash indicates the variant had already finished or had not yet reached that epoch. Bold marks the best (lowest) value reached by each variant.

| Epoch | V1 | V2 | V3 | V4 | V5 | V6 | V7 |
|---|---|---|---|---|---|---|---|
| 10 | 113.4 | 88.2 | 60.3 | 110.2 | 95.2 | 62.4 | 63.7 |
| 30 | 102.7 | 79.5 | 49.9 | 100.9 | – | 53.7 | 54.8 |
| 60 | 98.8 | 77.2 | 45.8 | 99.1 | 87.3 | 49.8 | 51.1 |
| 90 | 99.4 | 77.4 | **44.63** | 100.2 | – | 48.4 | 50.0 |
| 120 | 100.1 | 78.2 | 45.0 | 101.4 | – | 47.9 | 49.5 |
| 160 | – | – | – | – | 82.0 | 47.6 | 49.1 |
| 200 | – | – | – | – | 81.2 | **47.48** | 48.96 |
| 230 | – | – | – | – | – | – | **48.89** |
| 300 | 106.0 | 81.2 | 46.7 | 106.5 | 80.0 | – | – |
| 410 | – | – | – | – | **79.11** | – | – |
| *final* | 107.79 | 81.71 | 47.07 | 107.79 | 79.97 | 49.10 | 50.20 |